



UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes en la División de Ciencias y Artes para el Diseño (CyAD)

Proyecto Tecnológico

Reporte Final del Proyecto de Integración

Trimestre 2026-Primavera

Zuriel Godínez Ramos
2202003916
al2202003916@azc.uam.mx

M. en C Cesar Benavides Alvarez
Profesor Asociado
Departamento de Electrónica
cesarbenavidez@azc.uam.mx

M. en C Josué Figueroa González
Profesor Asociado
Departamento de Sistemas
jfgo@azc.uam.mx

11 de agosto de 2026

Declaratoria

Yo, César Benavides Álvarez, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

César Benavides Álvarez

Yo, Josué Figueroa González, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

Josué Figueroa González

Yo, Zuriel Godinez Ramos, doy mi autorización a la Coordinación de Servicios de Información de la Universidad Autónoma Metropolitana, Unidad Azcapotzalco, para publicar el presente documento en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco. En caso de que el Comité de Estudios de la Licenciatura en Ingeniería en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

Zuriel Godinez Ramos

Resumen

En la División de Ciencias y Artes para el Diseño (CyAD) de la Universidad Autónoma Metropolitana Unidad Azcapotzalco, los profesores entregan cada trimestre tres tipos de documentos: las *cartas temáticas* que describen el contenido de cada curso, los *requisitos de recuperación* para estudiantes en situación irregular y una *autoevaluación* única por trimestre de su propia labor docente. Hasta ahora este proceso se realizaba con formularios de Google y hojas de cálculo, lo que implicaba trabajo repetitivo, dificultades para dar seguimiento y una alta probabilidad de errores, pues muchos de los datos eran ingresados manualmente.

Para atender esta necesidad se desarrolló una aplicación web dividida en dos partes que se comunican entre sí: una parte visible en el navegador (frontend), con la que interactúan las personas usuarias, y una parte oculta que administra la información en una base de datos (backend). La aplicación se organiza alrededor de cinco funciones principales: acceso seguro con diferentes tipos de usuario (profesor, administrador y público general); captura y publicación de cartas temáticas; captura y publicación de requisitos de recuperación; llenado de autoevaluaciones con cálculo automático del nivel de desempeño; y generación de reportes de cumplimiento para la División.

Además de lo anterior se agregaron dos elementos que enriquecen la propuesta original: una vista pública que permite consultar los documentos sin necesidad de iniciar sesión, filtrando por trimestre, programa y unidad de enseñanza-aprendizaje, y si no hay documentos disponibles, se muestra el siguiente mensaje: "Solo se muestran UEA con al menos un documento publicado. Si una UEA no aparece es porque ningún profesor ha publicado su documento para este periodo"; y un panel de catálogos donde el personal administrativo mantiene actualizados los departamentos, licenciaturas, posgrados, áreas y unidades de enseñanza-aprendizaje (UEAs), esta última con posibilidad de importar información desde archivos CSV con la estructura adecuada.

En este reporte se describe el proceso de desarrollo, los resultados obtenidos, las diferencias entre lo comprometido y lo entregado — en particular la decisión de generar reportes en formato Excel en lugar de PDF — y las mejoras propuestas para versiones futuras del sistema.

Índice

Resumen	1
1. Introducción	7
2. Antecedentes	8
2.1. Google Forms en la evaluación diagnóstica como apoyo en las actividades docentes. Caso con estudiantes de la Licenciatura en Turismo [1].	8
2.2. La autoevaluación docente de aula: un camino para mejorar la práctica educativa [2].	8
2.3. Sistema para la configuración de formularios y captura de los resultados de la evaluación de Atributos de Egreso en un proceso de mejora continua [3].	8
2.4. Sistema académico de gestión escolar trimestralizado [4].	8
2.5. Sistema de Gestión Académica a través del desarrollo de Modelo-Vista-Controlador [5].	8
2.6. Sistema de Gestión Administrativa para la Escuela Secundaria No. 44 "Jorge Luis Borges"[6].	9
3. Justificación	11
4. Objetivos	12
4.1. Objetivo General	12
4.2. Objetivos Específicos	12
5. Marco teórico	13
5.1. Aplicación web y arquitectura cliente-servidor	13
5.2. Patrón Modelo-Vista-Controlador (MVC)	13
5.3. REST y APIs sobre HTTP	13
5.4. Python y Django	14
5.5. Django REST Framework (DRF)	14
5.6. PostgreSQL	15
5.7. React	15
5.8. Vite	16
5.9. Tailwind CSS	16
5.10. Bibliotecas complementarias del frontend	16
5.11. Autenticación con JSON Web Tokens (JWT)	17
5.12. Control de acceso basado en roles (RBAC)	17
5.13. Seguridad web y OWASP Top 10	18
5.14. Pruebas automatizadas	18
6. Desarrollo del proyecto	19
6.1. Arquitectura general	19
6.1.1. Estructura de directorios	20

6.2.	Modelo de datos	20
6.3.	Módulo 1: autenticación y gestión de usuarios	21
6.3.1.	Autenticación con JWT	21
6.3.2.	Gestión de usuarios	22
6.3.3.	Roles y permisos	22
6.4.	Módulo 2: gestión de cartas temáticas	22
6.4.1.	Ciclo de vida y CRUD	22
6.4.2.	Validaciones	22
6.4.3.	Interfaz de usuario	23
6.5.	Módulo 3: gestión de requisitos de recuperación	23
6.6.	Módulo 4: autoevaluación docente	23
6.6.1.	Form-builder para administradores	23
6.6.2.	Versionado de formularios	24
6.6.3.	Scoring ponderado	24
6.6.4.	Privacidad de respuestas	24
6.7.	Módulo 5: reportes de cumplimiento	24
6.8.	Catálogos administrables	25
6.9.	Vista pública de exploración	25
6.10.	Pruebas automatizadas	25
6.11.	Seguridad	26
7.	Resultados	27
7.1.	Métricas del código y de la base de datos	27
7.2.	Métricas de pruebas	28
7.3.	Evidencia visual del sistema en operación	28
7.3.1.	Autenticación	28
7.3.2.	Vista Docente	28
7.3.3.	Vista Administrador	28
7.3.4.	Vista pública	28
7.4.	Documentación de la API	28
8.	Análisis y discusión de resultados	29
8.1.	Comparación entregado vs. propuesta	29
8.2.	Discusión de las brechas frente a la propuesta	30
8.2.1.	Identidad por email en lugar de número económico	30
8.2.2.	Ausencia de exportación a PDF/DOCX en backend	30
8.2.3.	Cartas y requisitos aplanados	31
8.2.4.	Sin reset de contraseña autoservicio	31
8.3.	Elementos entregados que no estaban en la propuesta	31
8.4.	Limitaciones del sistema actual	31
9.	Conclusiones	33
9.1.	Cumplimiento por objetivo	33
9.2.	Aprendizajes técnicos	33
9.3.	Trabajo futuro	34

A. Código fuente	35
A.1. Estructura del repositorio	35
A.2. Backend — Modelo de Usuario con roles	35
A.3. Backend — JWT y throttling	36
A.4. Backend — Documento academico base	37
A.5. Backend — Scoring ponderado de autoevaluacion	37
A.6. Backend — Permisos por rol	38
A.7. Frontend — Cliente HTTP con refresh automatico de JWT	38
A.8. Frontend — Guard de rutas por rol	39
A.9. Frontend — Renderizador dinamico de preguntas	39
A.10.Backend — Comando de importacion CSV para UEA	40
A.11.Nota sobre el listado completo	40
B. Entregables del plan de trabajo	41

Índice de figuras

1.	Arquitectura de tres capas del sistema.	19
2.	Diagrama de casos de uso del sistema.	45
3.	Diagrama Entidad-Relación del sistema (agrupado por app Django).	46
4.	Pantalla de inicio de sesión.	46
5.	<i>Dashboard</i> del profesor: tarjetas de resumen y listas agrupadas por periodo.	47
6.	Lista de cartas temáticas del profesor.	47
7.	Formulario de creación/edición de carta temática.	48
8.	Formulario de requisitos de recuperación.	48
9.	Formulario de autoevaluación con renderizado dinámico de preguntas.	49
10.	<i>Dashboard</i> del administrador: KPIs globales y cumplimiento por departamento.	49
11.	Gestión de profesores: crear, editar, restablecer contraseña.	50
12.	Constructor de formularios de autoevaluación (<i>form-builder</i>).	50
13.	Panel de reportes con descarga en XLSX.	51
14.	Vista pública <i>Explorar</i> : cascada periodo→programa→UEA con acordeón de documentos.	51
15.	Vista pública de una carta temática publicada, con formato imprimible.	52

Índice de cuadros

1.	Comparación cualitativa de los trabajos relacionados con el proyecto propuesto.	10
2.	Métricas del código entregado.	27
3.	Volumen de pruebas por módulo.	28
4.	Grado de cumplimiento por componente respecto a la propuesta.	29
5.	Entregables prometidos en la propuesta y su estado real.	41

1. Introducción

En la Universidad Autónoma Metropolitana Unidad Azcapotzalco, particularmente en la División de Ciencias y Artes para el Diseño (CyAD), los profesores son responsables de llenar manualmente formularios en Google Forms que luego son capturados en hojas de cálculo para gestionar documentos clave como las cartas temáticas, las autoevaluaciones docentes y los requisitos de recuperación para alumnos. Este proceso, repetitivo y propenso a errores, dificulta la organización y el seguimiento adecuado de la información académica. Para resolver este problema, se propone el desarrollo de un sistema digital dirigido principalmente a los docentes, que automatice el llenado de datos académicos una vez que el usuario se haya identificado y seleccione el documento con el que va a trabajar. El sistema permitirá a los profesores: (1) gestionar cartas temáticas, (2) realizar su autoevaluación docente al finalizar el trimestre, y (3) subir requisitos de recuperación. Además, el sistema facilitará a los administradores el acceso a las actividades realizadas por los docentes y la generación de reportes sobre el cumplimiento de estas actividades.

2. Antecedentes

2.1. Google Forms en la evaluación diagnóstica como apoyo en las actividades docentes. Caso con estudiantes de la Licenciatura en Turismo [1].

Google Forms atiende la necesidad de recopilar, organizar y analizar información de manera eficiente. Permite crear encuestas y formularios sin conocimientos técnicos, con almacenamiento automático en Google Sheets e integración con otros servicios de Google. Además, permite utilizar plantillas prediseñadas o imágenes y logos propios.

2.2. La autoevaluación docente de aula: un camino para mejorar la práctica educativa [2].

El artículo aborda la temática de la autoevaluación docente como estrategia que permite recoger información relevante acerca del desempeño docente del aula. Se analiza la autoevaluación docente como una estrategia constructiva de mejora profesional. Se promueve la reflexión crítica sobre el desempeño en el aula, enfocándose en identificar fortalezas y áreas de oportunidad.

2.3. Sistema para la configuración de formularios y captura de los resultados de la evaluación de Atributos de Egreso en un proceso de mejora continua [3].

La evaluación de los Atributos de Egreso requiere información precisa en cada una de las evaluaciones. No obstante, al ser un proceso manual, es propenso a errores. El sistema automatiza la captura de información en procesos de evaluación académica. Incluye módulos que permiten precargar datos, validar usuarios y estructurar formularios según la UEA y los atributos a evaluar.

2.4. Sistema académico de gestión escolar trimestralizado [4].

Mediante un análisis preliminar realizado en la Unidad Educativa Técnico Humanístico "José Luis Suárez Guzmán" ubicada en la ciudad de El Alto, se observó que la institución no dispone de un sistema de monitoreo y registro virtual. El sistema propuesto centraliza la gestión académica, permitiendo consultar y registrar información pedagógica desde cualquier dispositivo. Mejora la trazabilidad y disponibilidad de los registros escolares.

2.5. Sistema de Gestión Académica a través del desarrollo de Modelo-Vista-Controlador [5].

La gestión administrativa suele manejar grandes volúmenes de datos, que sin el tratamiento correcto crean dificultades en las tareas. Este proyecto implementa módulos específicos para la gestión académica: planes de estudio, carga académica y disponibilidad docente. Utiliza una arquitectura Modelo-Vista-Controlador (MVC) para mejorar la organización del desarrollo.

2.6. Sistema de Gestión Administrativa para la Escuela Secundaria No. 44 "Jorge Luis Borges"[6].

El incremento significativo de la matrícula y del personal de la escuela produce un aumento en la cantidad de la documentación que se administra en forma manual, lo que origina errores en el registro de los datos. El sistema atiende el incremento de la carga administrativa, automatiza registros escolares y centraliza la información institucional. Fue construido con metodología XP, optimizando procesos en un entorno de cambio constante.

Las similitudes y diferencias con el proyecto propuesto se presentan en la Tabla 1.

Ref.	Similitudes	Diferencias
[1]	■ Uso de formularios digitales para capturar información. ■ No requiere conocimientos técnicos.	■ Permite trabajar individualmente o de forma colaborativa a distancia. ■ No tiene un sistema de autenticación con roles específicos.
[2]	■ Relevancia de la autoevaluación docente como herramienta de mejora. ■ Permitir que el docente identifique sus fortalezas y áreas de mejora.	■ Aborda la autoevaluación desde una perspectiva pedagógica, no tecnológica. ■ No plantea un sistema automatizado.
[3]	■ Captura estructurada de datos mediante formularios. ■ Autocompletado de formularios a partir de la información en una base de datos.	■ Enfocado en evaluar atributos de egreso por UEA. ■ Módulo para indicar los atributos de egreso a evaluar. ■ Módulo para asignar a un profesor a la UEA a evaluar.
[4]	■ Registro y visualización de información académica.	■ Orientado al seguimiento pedagógico de estudiantes, no a documentos docentes.
[5]	■ Módulos especializados para distintos procesos académicos. ■ Manejo de información de los planes de estudios.	■ Gestión enfocada en planes de estudio y disponibilidad docente, no en formularios.

Ref.	Similitudes	Diferencias
[6]	■ Centralización de datos. ■ Automatización de tareas administrativas.	■ Diseñado principalmente para el personal administrativo, no para profesores.

Tabla 1. Comparación cualitativa de los trabajos relacionados con el proyecto propuesto.

3. Justificación

El principal beneficio de este sistema es la optimización del tiempo, permitiendo un manejo más sencillo de la información que deben capturar los profesores. Además, esta solución contribuye a la modernización de la administración académica, alineándose con la tendencia de digitalización en las instituciones educativas [7]. Un sistema bien estructurado no solo beneficiará a los docentes, sino que también permitirá a la administración académica tener un mejor control y seguimiento de la información generada, promoviendo una gestión más eficiente y transparente.

4. Objetivos

4.1. Objetivo General

Desarrollar un sistema que permita a los profesores de la división de CyAD gestionar de manera eficiente sus datos académicos, la información sobre sus cursos impartidos, los requisitos de sus evaluaciones de recuperación, y su autoevaluación al final de cada trimestre.

4.2. Objetivos Específicos

1. Permitir el acceso al sistema mediante una autenticación que permita la identificación y uso del sistema según el rol del usuario.
2. Crear, modificar y consultar información sobre cartas temáticas y evaluaciones de recuperación por parte de los profesores.
3. Registrar información sobre la autoevaluación de los profesores al finalizar cada trimestre.
4. Crear reportes que faciliten al personal administrativo el seguimiento del cumplimiento de las actividades docentes.

5. Marco teórico

Esta sección define los conceptos, patrones arquitectónicos, protocolos y herramientas de software que sustentan el desarrollo del sistema. Cada concepto se acompaña de la justificación de por qué se seleccionó, cuál fue su papel dentro del proyecto y qué alternativas existían.

5.1. Aplicación web y arquitectura cliente–servidor

Una *aplicación web* es un programa cuya interfaz se ejecuta dentro de un navegador y cuya lógica principal reside en un servidor remoto. A diferencia de una aplicación de escritorio, no requiere instalación: basta con una URL. La arquitectura habitual es *cliente–servidor*, en la que dos programas independientes se comunican a través de una red: el *cliente* solicita datos u operaciones y el *servidor* las procesa y responde.

El sistema desarrollado adopta esta arquitectura con una separación estricta: el *frontend* (cliente) se ejecuta en el navegador del usuario y se distribuye como una *Single Page Application* (SPA); el *backend* (servidor) expone una API HTTP con la que el frontend se comunica mediante peticiones asíncronas. Esta separación permite desplegar y evolucionar cada capa de forma independiente, y facilita construir clientes adicionales (por ejemplo móviles) sobre la misma API.

5.2. Patrón Modelo–Vista–Controlador (MVC)

El *Modelo–Vista–Controlador* es un patrón arquitectónico introducido por Krasner y Pope para Smalltalk-80 [8]. Divide una aplicación con interfaz gráfica en tres responsabilidades:

- **Modelo:** representa los datos y las reglas de negocio. Es independiente de cómo se muestran o se manipulan.
- **Vista:** presenta el estado del modelo al usuario. Puede haber varias vistas de un mismo modelo.
- **Controlador:** recibe las entradas del usuario, decide qué operación realizar sobre el modelo y actualiza la vista.

En el proyecto la implementación es una particularización moderna: el *Modelo* corresponde a las clases de Django que se persisten en PostgreSQL; la *Vista* está formada por los componentes de React que se renderizan en el navegador; el *Controlador* se distribuye entre las *views* y *serializers* del backend (que traducen entre HTTP y el modelo) y los *hooks* y el *routing* del frontend (que reaccionan a los eventos del usuario).

5.3. REST y APIs sobre HTTP

REST (*Representational State Transfer*) es un estilo arquitectónico definido por Roy Fielding en su tesis doctoral [9]. Establece un conjunto de restricciones para diseñar servicios distribuidos escalables sobre HTTP:

- Cada recurso se identifica por una URI única (por ejemplo, `/api/v1/cartas-tematicas/42/`)
- Las operaciones se expresan con los verbos HTTP estándar: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (eliminar).
- Las respuestas usan una representación acordada (habitualmente JSON) y son *sin estado*: cada petición contiene toda la información necesaria para procesarla.
- Los códigos de estado HTTP (200, 201, 400, 401, 403, 404, etc.) comunican de manera estándar el resultado.

En el proyecto, todas las operaciones de negocio (crear una carta temática, publicar un requisito, enviar una autoevaluación, consultar un reporte) se exponen como recursos REST versionados bajo `/api/v1/`.

5.4. Python y Django

Python es un lenguaje de programación de alto nivel, interpretado y de tipado dinámico, ampliamente utilizado en desarrollo web, análisis de datos y automatización. Se seleccionó por su legibilidad, la madurez de su ecosistema y la disponibilidad de Django.

Django [10] es un *framework* web escrito en Python que sigue el principio “*baterías incluidas*”: provee, listos para usar, los componentes que la mayoría de las aplicaciones web necesitan. Los más relevantes para este proyecto son:

- **ORM (Object-Relational Mapper)**: permite definir el esquema de la base de datos como clases de Python (los modelos) y consultarla con métodos en lugar de SQL. Reduce errores de sintaxis, previene inyección SQL y hace portable el código entre motores de base de datos.
- **Sistema de migraciones**: cada cambio al modelo se traduce automáticamente en un archivo de migración versionable en git. Permite evolucionar la base de datos de forma reproducible entre entornos.
- **Sistema de autenticación**: usuarios, grupos, permisos y *hashing* de contraseñas ya vienen implementados; el proyecto extiende el modelo de usuario para agregar rol.
- **Panel de administración**: interfaz autogenerada para operar la base de datos, útil como respaldo del administrador.
- **Sistema de validación y formularios**: encapsula las reglas de negocio en el servidor.

5.5. Django REST Framework (DRF)

Django REST Framework [11] es una capa sobre Django que facilita construir APIs REST. Sus componentes principales son:

- **Serializers**: traducen entre objetos de Python (instancias de modelos) y representaciones serializables (JSON). Validan la entrada y controlan qué campos se exponen hacia afuera.

- **ViewSets y routers:** agrupan las operaciones CRUD sobre un recurso y las publican bajo URLs consistentes con las convenciones REST.
- **Permisos:** clases reutilizables que declaran quién puede acceder a cada acción (por ejemplo, “sólo el dueño del recurso puede modificarlo”).
- **Throttling:** limita el número de peticiones que un cliente puede hacer en un lapso, útil para mitigar ataques de fuerza bruta.
- **drf-spectacular:** complemento que analiza el código y genera automáticamente una descripción *OpenAPI* de la API, junto con un explorador *Swagger UI* interactivo.

En el proyecto DRF permite mantener la definición del contrato de la API cercana al modelo, con *serializers* explícitos que documentan qué campos son públicos y qué reglas de validación aplican.

5.6. PostgreSQL

PostgreSQL [12] es un sistema de gestión de bases de datos relacional (*RDBMS*), libre y de código abierto, con más de 30 años de desarrollo. Se seleccionó la versión 16 por varias razones:

- **Cumplimiento estricto del estándar SQL** y soporte de *transacciones ACID*, lo que garantiza consistencia de los datos incluso ante fallos.
- **Constraints declarativos avanzados:** *CHECK*, *UNIQUE*, *EXCLUDE*. En este proyecto se usan para forzar reglas como “una UEA pertenece a una licenciatura o a un posgrado, nunca a ambos” o “no puede haber dos UEA con la misma clave dentro del mismo programa”.
- **Tipos avanzados:** *JSONB* (JSON binario indexable), arreglos, tipos temporales precisos.
- **Compatibilidad con el ORM de Django:** no requiere adaptadores adicionales y aprovecha las funcionalidades específicas del motor.

5.7. React

React [13] es una biblioteca de JavaScript, desarrollada originalmente por Facebook (hoy Meta), para construir interfaces de usuario a partir de *componentes* reutilizables. El proyecto utiliza la versión 19.

Los conceptos centrales de React que se aprovechan son:

- **Componente:** función que recibe *props* (parámetros) y devuelve una descripción declarativa de la interfaz. Los componentes se componen entre sí formando árboles.
- **Estado (*state*):** datos internos de un componente que, al cambiar, disparan un nuevo renderizado.

- **Hooks:** funciones especiales (`useState`, `useEffect`, `useContext`, etc.) que permiten manejar estado y efectos secundarios en componentes funcionales.
- **Renderizado declarativo:** el programador describe cómo se ve la interfaz en función del estado; React se encarga de calcular y aplicar los cambios mínimos al DOM.

5.8. Vite

Vite [14] es una herramienta de *build* y servidor de desarrollo para aplicaciones frontend modernas. Sustituye a herramientas más antiguas como *webpack* y aprovecha los módulos nativos de JavaScript (*ES modules*) del navegador para arrancar el servidor en milisegundos, incluso en proyectos grandes.

Sus aportes en este proyecto son:

- **Servidor de desarrollo con *Hot Module Replacement*:** los cambios en el código se reflejan en el navegador sin recargar la página completa, preservando el estado.
- **Build optimizado para producción:** compila, minifica y divide el código en *chunks* para producir un paquete estático servible por cualquier servidor HTTP.
- **Configuración mínima** y ecosistema de *plugins*.

5.9. Tailwind CSS

Tailwind CSS [15] es un *framework* de estilos basado en *utility classes*: en lugar de escribir hojas de estilo (CSS) tradicionales, la apariencia se compone directamente en el marcado mediante clases atómicas (por ejemplo, `p-4`, `text-lg`, `bg-indigo-600`, `rounded-lg`).

Ventajas aprovechadas:

- Elimina la necesidad de nombrar clases semánticas cuando no aportan valor.
- El paso de *build* descarta las clases no utilizadas, produciendo hojas de estilo compactas.
- Aporta un sistema de diseño (espaciados, tipografía, paleta) coherente por defecto.
- Fomenta una interfaz responsiva mediante *breakpoint prefixes* (`sm:`, `md:`, `lg:`).

5.10. Bibliotecas complementarias del frontend

Alrededor de React se utilizan bibliotecas específicas para tareas concretas:

TanStack Query Gestor de estado de servidor. Se encarga de solicitar, cachear, revalidar e invalidar los datos que llegan del backend. Aporta reintento automático ante fallos transitorios, *polling* declarativo y sincronización entre pestañas.

React Hook Form Biblioteca de manejo de formularios que evita renderizados innecesarios y ofrece una API basada en *hooks*, ligera comparada con alternativas como *Formik*.

Zod Biblioteca de validación de esquemas con inferencia de tipos. Se combina con React Hook Form para validar en el cliente antes de enviar al servidor.

React Router Manejo del enrutamiento del lado del cliente. Permite construir SPAs con navegación sin recarga y *guards* de ruta según el rol.

axios Cliente HTTP con soporte de *interceptors*, utilizado para adjuntar automáticamente el *access token* JWT y refrescarlo cuando expira.

SheetJS (xlsx) Genera archivos *.xlsx* en el navegador para la descarga de reportes.

5.11. Autenticación con JSON Web Tokens (JWT)

Un *JSON Web Token* (JWT) [16] es un formato compacto y firmado criptográficamente para transmitir información entre partes. Se define en el estándar *RFC 7519* y consta de tres partes codificadas en Base64 y separadas por puntos: encabezado, *payload* (información del usuario y expiración) y firma (para verificar integridad).

En el proyecto la autenticación con JWT se implementa mediante *django-rest-framework-simplejwt* y sigue este flujo:

1. El usuario envía credenciales al endpoint `/api/v1/auth/login/`.
2. El servidor valida las credenciales y responde con dos tokens: uno de *acceso* (corta vida, típicamente 60 minutos) y uno de *refresco* (vida más larga, siete días).
3. El frontend envía el *access token* en el encabezado `Authorization: Bearer <token>` de cada petición posterior.
4. Cuando el *access token* expira, un *interceptor* del cliente HTTP solicita uno nuevo al endpoint `/api/v1/auth/refresh/` usando el *refresh token*.
5. Los *refresh tokens* rotan: cada renovación invalida el anterior [17].

Se prefirió JWT sobre el esquema tradicional de sesiones con *cookies* porque encaja mejor con APIs REST sin estado consumidas por un frontend desacoplado.

5.12. Control de acceso basado en roles (RBAC)

Role-Based Access Control (RBAC) es un modelo de autorización en el que los permisos no se asignan a usuarios individualmente sino a *roles*, y cada usuario recibe uno o varios roles. El proyecto define dos roles:

- **ADMIN:** gestiona catálogos, profesores, formularios de autoevaluación y consulta reportes globales.
- **PROFESOR:** gestiona sus propias cartas temáticas, requisitos y autoevaluaciones.

Los permisos se aplican en dos capas:

- En el **backend** mediante clases *DRF permission* (por ejemplo, `IsOwnerProfessorOrAdminReadOn`) que verifican rol y propiedad del recurso.
- En el **frontend** mediante *guards* de ruta que redirigen si el rol no coincide, y renderizado condicional que oculta acciones no autorizadas.

La verificación en ambos lados es defensa en profundidad: el backend es la fuente de verdad; el frontend evita mostrar interfaces inútiles.

5.13. Seguridad web y OWASP Top 10

El *OWASP Top 10* [18] es un documento consensuado por la *Open Web Application Security Project* que enumera las diez categorías de riesgo de seguridad más críticas en aplicaciones web. Sirve como *checklist* de referencia para desarrolladores. Las categorías atendidas explícitamente en este proyecto son:

- **A01 — Broken Access Control:** se mitiga con las verificaciones de rol y propiedad en el backend.
- **A02 — Cryptographic Failures:** contraseñas hasheadas con PBKDF2 (algoritmo por defecto de Django) y tokens firmados con HMAC-SHA256.
- **A03 — Injection:** prevenida por el ORM (consultas parametrizadas) y por la validación de entrada en *serializers*.
- **A05 — Security Misconfiguration:** en producción se establece `DEBUG=False` y una *whitelist* explícita de *hosts* y de orígenes CORS.
- **A07 — Identification and Authentication Failures:** se aplica *throttling* en el *login* (5 intentos por minuto), rotación de *refresh tokens* y política de contraseñas fuerte.

5.14. Pruebas automatizadas

Las pruebas automatizadas del proyecto se ejecutan con *pytest* y *pytest-django*. Los conceptos utilizados son:

- **Prueba unitaria:** verifica una función o unidad aislada.
- **Prueba de integración:** verifica la interacción de varios componentes (por ejemplo, una petición HTTP que atraviesa *view*, *serializer*, ORM y base de datos).
- **Factory:** patrón para construir instancias de prueba consistentes; se emplea la biblioteca *factory-boy*.
- **Cobertura de código:** porcentaje del código ejecutado por las pruebas. Se mide con *coverage.py* a través del complemento *pytest-cov*.

Se privilegian las pruebas de integración sobre las unitarias porque atrapan errores en las fronteras (URL, permisos, serialización) que son las más frecuentes en una API REST.

6. Desarrollo del proyecto

Esta sección describe la arquitectura general del sistema, su modelo de datos, cada uno de los cinco módulos funcionales entregados y los aspectos transversales de *testing* y seguridad. La estructura del código se divide en dos monorepositorios lógicos: el `backend/` (Django) y el `frontend/` (React + Vite), ambos versionados con *git* bajo el directorio raíz `proyecto_terminal_cyad/`.

6.1. Arquitectura general

El sistema sigue una arquitectura cliente-servidor de tres capas:

1. **Capa de presentación (frontend):** aplicación SPA en React 19 servida por Vite 8; renderizada en el navegador; consume la *API REST* mediante *axios*; almacena el *JWT* en memoria y gestiona el ciclo de vida de las sesiones (*refresh* automático, expulsión al expirar).
2. **Capa de aplicación (backend):** servidor Django 5 que expone la *API REST* bajo el prefijo `/api/v1/`; orquesta la lógica de negocio en *ViewSets* de DRF; valida y serializa entradas con *Serializers*; documenta la API con *drf-spectacular* (`/api/docs/`).
3. **Capa de persistencia:** base de datos PostgreSQL 16 accedida vía el ORM de Django, con migraciones versionadas por app.

La Figura 1 muestra el diagrama de arquitectura y la Figura 2 el diagrama de casos de uso, con los actores *Administrador*, *Profesor* y *Público* y sus interacciones con los cinco módulos funcionales.

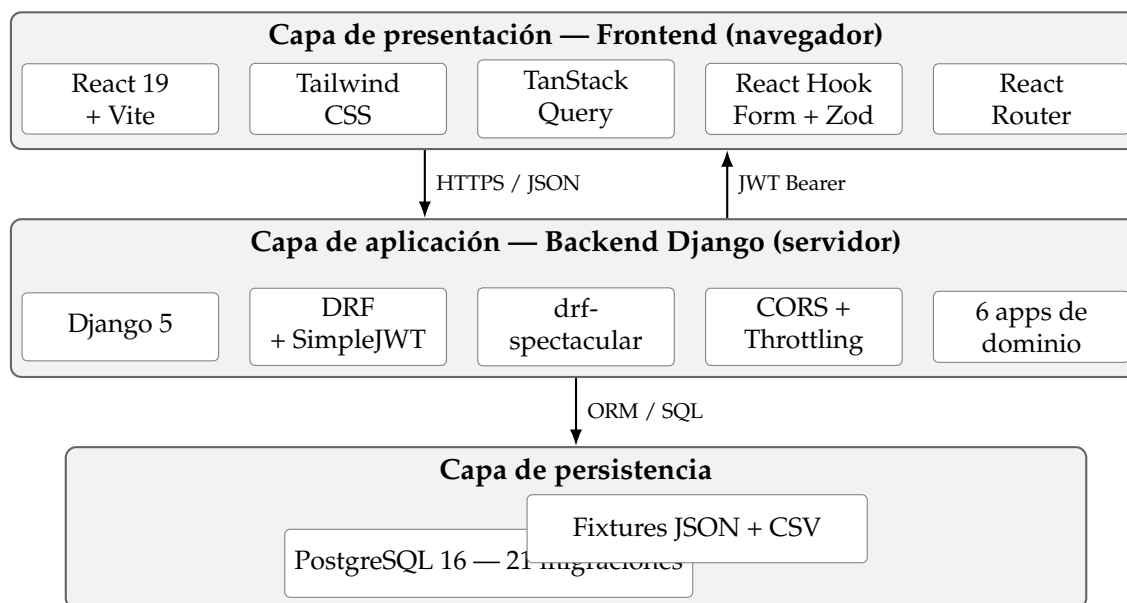


Figura 1. Arquitectura de tres capas del sistema.

6.1.1. Estructura de directorios

- **backend/**: proyecto Django. Contiene la configuración global (*config/*), las seis apps (*core*, *accounts*, *catalogos*, *documentos*, *autoevaluacion*, *reportes*), *scripts* de *seeding* (*scripts/*), suite de pruebas (*tests/*) y *requirements.txt*.
- **frontend/**: aplicación React + Vite. Contiene *src/pages/* (por rol: *admin/*, *profesor/*, *publico/*), *src/components/ui/* (componentes reutilizables), *src/api/* (clientes HTTP), *src/auth/* (contexto de sesión y *guards*) y *src/layouts/*.
- **docs/**: documentación complementaria; contiene este reporte (*docs/reporte/*) y los manuales (*docs/manuales/*).

6.2. Modelo de datos

El modelo de datos se distribuye por app siguiendo la separación de dominios. Un diagrama entidad-relación completo se muestra en la Figura 3.

Las entidades principales por app son:

core Contiene dos clases abstractas reutilizables: *TimeStampedModel* (con *created_at* y *updated_at*) y *EstadoActivoModel* (con estado booleano). No define tablas propias.

accounts Define Usuario (extiende *AbstractBaseUser*), con *email* como *USERNAME_FIELD*, *nombre*, *rol* {ADMIN, PROFESOR}, *is_active* y *is_staff*. La entidad Profesor extiende al usuario con un perfil *OneToOne* que agrega *numero_economico* (único), *nombre_completo*, *correo_institucional* y FK a Departamento.

catalogos Encapsula los catálogos institucionales: Departamento (*clave*, *nombre*), Licenciatura y Posgrado (*clave*, *nombre*, FK a departamento), Area (*nombre*, *descripción*), UEA (*clave*, *nombre*, trimestre 1–12, tipo {OBL, OPT, OTRO}, créditos, *liga*; FK a *Licenciatura XOR Posgrado*, FK a *Área*; *clave* única por programa) y Periodo (*clave* YY-I/P/O, *fecha_inicio*, *fecha_fin* y tres *flags* de recurso activo: *activo_cartas*, *activo_requisitos*, *activo_autoevaluacion*).

documentos Contiene una clase abstracta *DocumentoAcademicoBase* con los campos comunes (FK a Profesor y UEA, FK a Periodo, *nombre_grupo*, *id_grupo*, *horario*, *modalidad* y estado {BORRADOR, PUBLICADO}) y *snapshots* de identidad del profesor (*profesor_nombre*, *profesor_correo*) que preservan la autoría aunque el usuario sea desactivado. De ella heredan dos modelos concretos:

- **CartaTematica**: campos textuales libres (*descripcion_uea*, *objetivo_general*, *objetivos_particulares*, *contenido_sintetico*, *objetivos_aprendizaje*, *requerimientos*, *conocimientos_previos*, *modalidad_evaluacion*, *revisiones_asesorias*, *bibliografia*, *enlace*, *calendarizacion_actividades*), con unicidad por (*profesor*, *periodo*, *uea*, *id_grupo*).
- **RequisitoRecuperacion**: *lugar*, *duracion_aprox*, *fecha_hora*, *recursos_necesarios*, *requisitos*, *notas*, con misma unicidad.

autoevaluacion Modela el *form-builder* y las respuestas: *Formulario* (título, descripción, FK Periodo, estado {BORRADOR, PUBLICADO, CERRADO}, versión, bandera *una_respuesta_por_profesor*); *Seccion* (título, orden, peso decimal cuya suma es 100 %); *Pregunta* (ocho tipos: TEXTO_CORTO, TEXTO_LARGO, OPCION_UNICA, CASILLAS, SI_NO, ESCALA_LINEAL, LISTA_DESPLEGABLE, CUADRICULA; campo *config* en JSON); *OpcionPregunta* (texto, valor, puntos); *FilaCuadrícula* *NivelDesempeno* (rango porcentaje_min-porcentaje_max, observación, color); *Respuesta* (FK Formulario y Profesor, versión, estado {BORRADOR, ENVIADO}, *puntaje_obtenido*, porcentaje, FK *NivelDesempeno*); *RespuestaPregunta*, *RespuestaCelda* y *RespuestaSeccion* (snapshot de puntaje por sección al enviar).

reportes No define modelos propios; sólo agrega vistas de agregación sobre las entidades de otras apps mediante consultas ORM.

En total, el sistema cuenta con 21 migraciones aplicadas (*accounts:3, autoevaluacion:5, catalogos:7, documentos:5, core y reportes:1* cada uno para `__init__`). La migración `0002_rediseño_documentos` de la app *documentos* materializa la decisión de aplanar los documentos (eliminar los submodelos *Tema*, *Subtema*, *Bibliografía*, *CriterioEvaluacion* propuestos originalmente).

6.3. Módulo 1: autenticación y gestión de usuarios

Este módulo, implementado en la app *accounts*, controla el acceso al sistema y la gestión del ciclo de vida de los usuarios.

6.3.1. Autenticación con JWT

El *endpoint* `POST /api/v1/auth/login/` extiende *TokenObtainPairView* de *simplejwt* con un serializer personalizado que:

- Diferencia *credenciales inválidas* de *cuenta desactivada* en los mensajes de error, para dar diagnóstico útil sin filtrar existencia de cuentas.
- Emite dos tokens: un *access token* con vigencia de 60 minutos y un *refresh token* con vigencia de 7 días. Los *refresh tokens* se rotan en cada uso.
- Aplica `throttle_scope="login"` para limitar a cinco intentos por minuto por dirección IP (véase § 6.11).

El *endpoint* `POST /api/v1/auth/refresh/` intercambia un *refresh token* válido por un nuevo par de tokens. El cliente frontend intercepta las respuestas HTTP 401 y dispara automáticamente el *refresh* antes de reintentar la petición original (`frontend/src/api/client.js`).

6.3.2. Gestión de usuarios

El `UsuarioViewSet` expone las operaciones CRUD sobre usuarios, restringidas al rol *Administrador* mediante una *permission class* personalizada. La acción `POST /usuarios/{id}/set-password/` permite al administrador restablecer la contraseña de cualquier profesor.

El `ProfesorViewSet` expone las operaciones sobre el perfil de `Profesor` y admite filtrado por departamento y búsqueda por nombre o número económico.

6.3.3. Roles y permisos

Los dos roles *ADMIN* y *PROFESOR* se materializan como valores de un `TextChoices` en el modelo `Usuario`. La comprobación de rol se realiza:

- En backend, mediante *permission classes* de DRF (`IsOwnerProfesorOrAdminReadOnly`, `IsAdminRole`) que se declaran a nivel de `ViewSet`.
- En frontend, mediante los *wrappers* de ruta `AdminRoute` y `ProfesorRoute` (`src/auth/RoleRoute`) que redirigen a `/login` si el rol no coincide.

6.4. Módulo 2: gestión de cartas temáticas

Implementado en la app `documentos`, este módulo permite al profesor gestionar sus cartas temáticas del trimestre.

6.4.1. Ciclo de vida y CRUD

El `CartaTematicaViewSet` expone las operaciones estándar sobre `/api/v1/cartas-tematicas/` con `IsOwnerProfesorOrAdminReadOnly`: cada profesor sólo ve y edita sus propias cartas; el administrador tiene acceso de solo lectura a todas las cartas. La acción personalizada `POST /cartas-tematicas/{id}/cambiar-estado/` transiciona el documento entre *BORRADOR* y *PUBLICADO*.

6.4.2. Validaciones

Además de la unicidad (*profesor*, *periodo*, *uea*, *id_grupo*) declarada como *constraint*, el serializer valida que:

- Sólo se puedan crear cartas para `Periodos` con `activo_cartas=True`.
- El profesor sólo pueda registrar cartas para UEAs de licenciaturas activas.
- Los *snapshots* `profesor_nombre` y `profesor_correo` se llenen automáticamente al crear el documento.

6.4.3. Interfaz de usuario

La interfaz del profesor consta de tres pantallas: lista (`CartasListPage`), formulario de creación/edición (`CartaFormPage`) y vista previa (`CartaPreviewPage`). El listado se agrupa por periodo mediante el componente `CollapsiblePeriodoCard` y muestra un *badge* de estado. La interfaz del administrador (`CartasAdminPage`) muestra la vista global filtrable por periodo, con enlaces a la vista pública de cada carta publicada.

6.5. Módulo 3: gestión de requisitos de recuperación

Este módulo comparte código y patrones con el de cartas temáticas: mismo *base abstract model*, mismos permisos, mismo ciclo BORRADOR/PUBLICADO, misma unicidad. La diferencia está en los campos específicos —lugar, fecha_hora, duracion_aprox, recursos_necesarios, requisitos y notas— y en la bandera de periodo activo_requisitos, independiente de la de cartas: esto permite que la Coordinación abra la ventana de captura de cartas y la de requisitos en momentos distintos del trimestre.

6.6. Módulo 4: autoevaluación docente

Éste es el módulo más elaborado del sistema. Se implementó en nueve incrementos etiquetados como *PR1* a *PR9* en el historial de git.

6.6.1. Form-builder para administradores

El administrador diseña formularios de autoevaluación desde una interfaz drag-and-drop (`FormularioBuilderPage`) que soporta ocho tipos de pregunta:

1. Texto corto (una línea).
2. Texto largo (párrafo).
3. Opción única (radio).
4. Casillas (checkboxes múltiples).
5. Sí/No.
6. Escala lineal (por ejemplo 1–5 tipo Likert).
7. Lista desplegable.
8. Cuadrícula (matriz filas $\times$ columnas para preguntas Likert compactas).

Cada pregunta pertenece a una `Seccion` con un peso decimal; la suma de los pesos de las secciones de un formulario debe ser exactamente 100 %. Cada `OpcionPregunta` tiene un campo `puntos`, y los `NivelDesempeno` definen rangos porcentuales (por ejemplo 0–40 *Insuficiente*, 40–70 *Suficiente*, 70–90 *Bueno*, 90–100 *Sobresaliente*) con una *observación* textual que se despliega al profesor al enviar su respuesta.

6.6.2. Versionado de formularios

Los formularios en estado *PUBLICADO* son inmutables: para modificar preguntas ya publicadas, el administrador debe *publicar una revisión*, lo que incrementa el número de versión del formulario. Las respuestas previas conservan la referencia a la versión con la que fueron enviadas (*version_formulario*), garantizando integridad estadística e histórica. El sistema también permite duplicar un formulario para crear una nueva versión desde cero.

6.6.3. Scoring ponderado

Cuando el profesor envía una respuesta, la vista `RespuestaViewSet.enviar` calcula:

1. El puntaje obtenido por sección como suma de puntos de las opciones marcadas.
2. El puntaje máximo posible por sección.
3. El porcentaje ponderado global, ponderando por el peso de cada sección.
4. Almacena un `RespuestaSeccion` por sección con estos totales, para permitir análisis posterior a nivel de sección.
5. Determina el `NivelDesempeno` correspondiente al porcentaje global y adjunta su *observación*.

6.6.4. Privacidad de respuestas

El administrador puede ver quiénes respondieron cada formulario y el porcentaje agregado por profesor, así como estadísticas descriptivas por sección, pero las respuestas individuales cualitativas se marcan como confidenciales del profesor. Esta separación se materializa mediante dos *serializers* distintos: uno *summary* para el administrador y uno *detail* para el propio profesor.

6.7. Módulo 5: reportes de cumplimiento

La app `reportes` expone cinco endpoints de solo lectura, todos restringidos al rol *Administrador*:

GET `/api/v1/reportes/dashboard/` Métricas globales: total de profesores activos, cartas publicadas, requisitos publicados, respuestas de autoevaluación enviadas, y tasas de cumplimiento por periodo.

GET `/api/v1/reportes/cumplimiento/` Detalle de cumplimiento por departamento: para cada departamento, cuenta profesores con al menos una carta publicada y al menos un requisito publicado en el periodo activo.

GET `/api/v1/reportes/cumplimiento-licenciatura/` Idem, agrupado por licenciatura.

GET /api/v1/reportes/autoevaluacion-profesores/ Lista los profesores con su porcentaje agregado, indicando quién ya envió y quién no.

GET /api/v1/reportes/resumen-autoevaluacion/ Estadísticas descriptivas por formulario: total de respuestas, porcentaje promedio, distribución por nivel de desempeño.

En el frontend, la `ReportesPage` presenta estos datos en tableros interactivos y ofrece descarga en formato *XLSX* generado en el cliente con *SheetJS* (*xlsx* v0.18.5). La exportación es unificada: el administrador elige tipo de reporte y periodo, y descarga un libro con las hojas correspondientes.

6.8. Catálogos administrables

La app `catalogos` expone *ViewSet* de CRUD completo para `Departamento`, `Licenciatura`, `Posgrado`, `Area`, `UEA` y `Periodo`, todos restringidos al rol *Administrador*. Adicionalmente, el comando de gestión `python manage.py cargar_catalogos_csv` realiza importación idempotente desde archivos CSV para las UEAs (típicamente muchas por programa), evitando la captura manual masiva.

Los *fixtures* JSON incluidos (`catalogos/fixtures/*.json`) precargan los cuatro departamentos de CyAD, las tres licenciaturas activas, los posgrados iniciales y las áreas de conocimiento, lo cual reduce el tiempo de puesta en marcha del sistema a minutos.

6.9. Vista pública de exploración

Este componente —no comprometido en la propuesta original y añadido a solicitud posterior— permite a cualquier persona consultar cartas y requisitos publicados sin autenticarse. La página `ExplorarPage` implementa una cascada de tres pasos: (1) selección de periodo, (2) selección de licenciatura o posgrado, (3) navegación por UEA agrupada por trimestre y área con un acordeón que muestra los documentos publicados por cada UEA. Al hacer clic en un documento se abre una vista imprimible (`PublicCartaPage` o `PublicRequisitoPage`) con `window.print()` disponible como acción principal.

Los *endpoints* públicos correspondientes (bajo el prefijo `/api/v1/publico/`) no requieren autenticación y sólo devuelven documentos en estado *PUBLICADO*.

6.10. Pruebas automatizadas

El backend cuenta con una suite exhaustiva de pruebas construida con *pytest* 8, *pytest-django* 4 y *factory-boy* 3. La distribución aproximada de líneas de test por app es:

- `tests/test_auth.py`: 491 líneas.
- `tests/test_autoevaluacion.py`: 2 044 líneas.
- `tests/test_catalogos.py`: 563 líneas.
- `tests/test_documentos.py`: 698 líneas.

- `tests/test_reportes.py`: 390 líneas.
- **Total**: aproximadamente 4 186 líneas.

Adicionalmente, `backend/conftest.py` incluye una *fixture autouse* que limpia el *cache* de Django antes de cada prueba, evitando que el *throttling* acumulado del *login* contamine tests posteriores. La ejecución típica de la suite completa toma unos minutos y no requiere infraestructura adicional (usa una base de datos de test aislada).

Los detalles del reporte de cobertura se presentan en la Sección 7.

6.11. Seguridad

Las medidas de seguridad implementadas siguen las recomendaciones de *OWASP Top 10* [18]:

- **Modo DEBUG seguro por defecto**: `DEBUG=False` salvo que `.env` lo active explícitamente (`config/settings.py:23`), evitando la exposición accidental de trazas y variables en producción.
- **Rate limiting del login**: cinco intentos por minuto vía `ScopedRateThrottle` (`settings.py:1`). Después del quinto intento fallido, el endpoint responde 429 Too Many Requests.
- **Validación de contraseñas**: se aplican los cuatro validadores estándar de Django (similitud con datos de usuario, longitud mínima, contraseñas comunes, contraseñas numéricas).
- **Hashing de contraseñas**: por defecto Django usa PBKDF2 con SHA-256.
- **CORS whitelist**: los orígenes permitidos se configuran vía `CORS_ALLOWED_ORIGINS` y se envían credenciales solo con los que estén en la lista.
- **Autenticación por JWT rotativo**: el *refresh* rota en cada uso, minimizando el impacto de una filtración.
- **Diferenciación de rol y permisos por ViewSet**: no hay endpoints sin permiso explícito; el default global es `IsAuthenticated`.
- **Endpoints públicos aislados**: bajo el prefijo `/api/v1/publico/`, expuestos con permiso `AllowAny` pero limitados a lectura de documentos en estado *PUBLICADO*.

7. Resultados

Esta sección presenta los resultados obtenidos con evidencia cuantitativa (métricas del código, tests, cobertura y endpoints) y evidencia cualitativa (capturas de pantalla del sistema en operación con datos de *seed*).

7.1. Métricas del código y de la base de datos

La Tabla 2 resume el volumen del sistema entregado.

Tabla 2. Métricas del código entregado.

Componente	Cantidad	Notas
Apps Django	6	core, accounts, catalogos, documentos, autoevaluacion, reportes
Migraciones aplicadas	21	Distribuidas entre las apps de dominio
Endpoints REST de la API v1	~45	Documentados automáticamente vía OpenAPI en /api/docs/
Modelos concretos	14	Sin contar clases abstractas TimeStampedModel y DocumentoAcademicoBase
Dependencias Python	11	Ver backend/requirements.txt
Dependencias JavaScript (prod)	11	Ver frontend/package.json
Tipos de pregunta soportados	8	TEXTO_CORTO, TEXTO_LARGO, OPCION_UNICA, CASILLAS, SI_NO, ESCALA_LINEAL, LISTA_DESPLEGABLE, CUADRICULA
Componentes UI reutilizables	10+	Button, Alert, Badge, Table, Modal, FormField, Loading, EmptyState, CollapsiblePeriodoCard, entre otros
Fixtures precargados	5	Departamentos, licenciaturas, posgrados, áreas, periodos (catalogos/fixtures/)

7.2. Métricas de pruebas

La Tabla 3 muestra el volumen de la suite de pruebas del backend.

Tabla 3. Volumen de pruebas por módulo.

Archivo de pruebas	Líneas	Cobertura módulo
tests/test_auth.py	491	Autenticación y usuarios
tests/test_autoevaluacion.py	2 044	Autoevaluación (form-builder + scoring)
tests/test_catalogos.py	563	Catálogos institucionales
tests/test_documentos.py	698	Cartas y requisitos
tests/test_reportes.py	390	Reportes de cumplimiento
Total backend	≈4 186	

La ejecución completa de la *suite* arroja **192 pruebas exitosas de 196** (98 %) y una **cobertura de código del 93 %** medida con `coverage.py` (comando `pytest -cov=. -cov-report=html`). Las cuatro pruebas restantes corresponden a casos avanzados del módulo de reportes de autoevaluación (cálculo de tasa de respuesta y estructura de *payload*) que quedan documentadas como trabajo futuro en la Sección 8. El reporte HTML detallado se incluye en la carpeta de entregables bajo `cobertura/`.

7.3. Evidencia visual del sistema en operación

Las siguientes capturas se tomaron con el sistema ejecutándose localmente contra la base de datos de *seed* (`python scripts/seed_demo.py`). Las credenciales usadas son las del entorno de desarrollo (`admin@cyad.uam.mx / Admin1234!` para el administrador; un profesor de prueba también sembrado).

7.3.1. Autenticación

7.3.2. Vista Docente

7.3.3. Vista Administrador

7.3.4. Vista pública

7.4. Documentación de la API

La API queda documentada automáticamente vía *drf-spectacular*. En `/api/docs/` se accede a la interfaz *Swagger UI* interactiva, y en `/api/schema/` al esquema OpenAPI en YAML descargable.

8. Análisis y discusión de resultados

Esta sección compara lo entregado con lo comprometido en la propuesta, discute las decisiones de diseño que se apartaron de aquélla y examina las limitaciones del sistema actual.

8.1. Comparación entregado vs. propuesta

La Tabla 4 sintetiza el grado de cumplimiento de cada componente del sistema respecto de lo que se prometió en la propuesta.

Tabla 4. Grado de cumplimiento por componente respecto a la propuesta.

Componente	Cumplimiento	Observaciones
Módulo de autenticación	Completo con matices	Identidad por email en lugar de número económico; <i>rate limiting</i> sí; <i>reset</i> autoservicio no.
Cartas temáticas (CRUD)	Completo	Ciclo Borrador/Publicado, unicidad, permisos, snapshots de identidad.
Cartas temáticas (contenido estructurado)	Reemplazado	Los submodelos Tema/Subtema/Bibliografía/CriterioEvaluacion se reemplazaron por texto libre tras el rediseño de junio 2026.
Requisitos de recuperación (CRUD)	Completo	Mismo patrón que cartas.
Requisitos de recuperación (RequisitoItem)	Reemplazado	También aplanado en el rediseño.
Exportación a PDF/-DOCX	No entregado	Se implementó exportación a XLSX en el cliente para reportes; documentos usan <code>window.print()</code> del navegador.
Autoevaluación docente (formulario)	Ampliado	Se entregó un <i>form-builder</i> configurable con ocho tipos de pregunta y ocho iteraciones (PR1–PR9).
Autoevaluación (scoring y observaciones)	Completo	<i>Scoring</i> ponderado por sección con niveles de desempeño automáticos.
Autoevaluación (PDF descargable por el docente)	No entregado	El resultado se muestra en pantalla; no hay descarga en PDF.
Autoevaluación (privacidad admin)	Ajustado	El admin ve porcentajes agregados y quién respondió; el detalle cualitativo queda para el propio profesor.

Componente	Cumplimiento	Observaciones
Reportes de cumplimiento	Completo	Dashboards por departamento y por licenciatura; export XLSX.
Reportes en PDF	No entregado	Ver observación de exportación.
Base de datos PostgreSQL	Completo	21 migraciones, <i>constraints</i> declarativos, fixtures y CSV para catálogos.
Autenticación JWT + validación de contraseñas + CORS whitelist	Completo	Ver Sección 6.11.
Manual de usuario y de instalación	Entregados	En docs/manuales/ (documentos separados).
Vista pública de exploración	Extra (no comprometido)	Cascada periodo→programa→UEA.
Catálogos administrables completos	Extra (no comprometido)	Departamentos, licenciaturas, posgrados, áreas, UEA (con CSV) y periodos.

8.2. Discusión de las brechas frente a la propuesta

8.2.1. Identidad por email en lugar de número económico

La propuesta indicaba autenticación con número económico + contraseña. En el sistema entregado, el `USERNAME_FIELD` es `email` y el número económico existe como campo opcional del perfil `Profesor`. La decisión se sostuvo por tres razones: (a) el email institucional es único, garantizado y conocido por todo el personal; (b) el número económico no siempre está disponible en el momento del alta y varía menos entre trámites; y (c) el email facilita la comunicación (envío de correos institucionales, futuras notificaciones, reset de contraseña). El impacto para el usuario es mínimo: en la práctica ingresa su email institucional en lugar de su número.

8.2.2. Ausencia de exportación a PDF/DOCX en backend

Ésta es la brecha de mayor visibilidad. La propuesta prometía generación de PDFs y DOCXs para cartas, requisitos, resultados de autoevaluación y reportes. En el sistema entregado, la única forma de *generar un PDF* de una carta o requisito es abrir la vista pública e imprimir a PDF desde el navegador (`window.print()`); la única exportación estructurada es la descarga a XLSX de los reportes, generada en el cliente con *SheetJS*. Las razones prácticas fueron: (a) evitar la dependencia de librerías pesadas (*WeasyPrint* requiere GTK; *ReportLab* requiere maquetación manual); (b) priorizar el flujo de operación diaria (captura, publicación, reportes en línea) sobre entregables descargables; y (c) concentrar el

esfuerzo en el módulo de autoevaluación, técnicamente más complejo. En la Sección 9 se marca la generación nativa de PDFs como trabajo futuro prioritario.

8.2.3. Cartas y requisitos aplanados

Los submodelos `Tema`, `Subtema`, `Bibliografia` y `CriterioEvaluacion` propuestos originalmente para estructurar el contenido de la carta temática se eliminaron mediante la migración `documentos/0002_rediseño_documentos` y se sustituyeron por campos `TextField` libres. La motivación fue la ausencia de una plantilla oficial de CyAD que estableciera con carácter normativo qué campos debían existir y con qué cardinalidad. Con contenido libre, cada profesor organiza su carta como considere conveniente, y la Coordinación puede definir posteriormente un formato estándar sin necesidad de migrar datos.

8.2.4. Sin reset de contraseña autoservicio

Sólo el administrador puede restablecer la contraseña de un profesor. Un flujo de *forgot password* con token por correo se documenta como línea de trabajo futuro pero no se entregó porque el envío de correos requiere infraestructura adicional (SMTP institucional configurado) que no estuvo disponible en el trimestre de desarrollo.

8.3. Elementos entregados que no estaban en la propuesta

Estos componentes se incorporaron a solicitud durante el desarrollo y elevan el valor del sistema:

- **Vista pública *Explorar*:** cascada periodo→programa→UEA para consulta de cartas y requisitos publicados. Beneficia a estudiantes y personal externo.
- **Form-builder de autoevaluación:** en lugar de un formulario fijo, el administrador diseña formularios con ocho tipos de pregunta, secciones ponderadas y versionado.
- **Catálogos administrables completos** con importación CSV.
- **Snapshots de identidad del profesor** en los documentos, para preservar autoría aunque el registro sea desactivado.
- **Suite de pruebas de ~4 200 líneas** en el backend.

8.4. Limitaciones del sistema actual

- **Sin pruebas en el frontend:** no hay *Vitest*, *Jest* ni *React Testing Library* configurados. Los flujos críticos se validan sólo por inspección manual y por los tests de la API que consumen.
- **Sin CI/CD:** no hay pipeline configurado; las pruebas y *lints* se ejecutan manualmente antes de cada *commit*.

- **Sin backups automáticos configurados:** la política de respaldo dependerá de la infraestructura de despliegue.
- **Sin métricas de uso:** no se registra actividad de usuarios para análisis posterior (no hay dashboard de *engagement* ni logs estructurados).
- **Sin accesibilidad medida:** aunque *Tailwind* y los componentes reutilizables favorecen la accesibilidad, no se realizó una auditoría formal con *Lighthouse* o *axe-core*.

Estas limitaciones no impiden la operación del sistema pero delimitan expectativas y sugieren prioridades para iteraciones futuras.

9. Conclusiones

El proyecto cumplió con el objetivo general planteado en la propuesta: se desarrolló e implementó un sistema web funcional que permite a los profesores de la División de Ciencias y Artes para el Diseño (CyAD) de la UAM-A gestionar cartas temáticas, requisitos de recuperación y responder autoevaluaciones, con acceso diferenciado por rol y reportes de cumplimiento para el personal administrativo. Los cuatro objetivos específicos se cumplieron sustancialmente, con matices que se detallan en la Sección 8 y que sintetizamos aquí.

9.1. Cumplimiento por objetivo

1. **Autenticación con roles:** cumplido. El sistema autentica mediante *JWT*, diferencia dos roles y aplica *throttling* sobre el *login*. El único matiz es que la identidad primaria del usuario es el `email` y no el número económico, decisión de diseño explicada en la Sección 8.
2. **CRUD de cartas y requisitos:** cumplido en cuanto a las operaciones y al ciclo Borrador/Publicado. La exportación a PDF y a DOCX —prometida en la propuesta— se sustituyó por la impresión nativa del navegador (`window.print()`) en la vista pública, sin generación de archivos binarios estructurados en el backend.
3. **Autoevaluación docente:** cumplido y ampliado. El sistema entregado va más allá de un formulario fijo: incorpora un *form-builder* configurable por el administrador, ocho tipos de pregunta, *scoring* ponderado por sección, niveles de desempeño automáticos y versionado de formularios que preserva la integridad histórica.
4. **Reportes de cumplimiento:** cumplido en cuanto a los indicadores. La exportación se implementó en formato *XLSX* en el cliente en lugar de *PDF* en el servidor.

9.2. Aprendizajes técnicos

Los principales aprendizajes derivados del desarrollo se enuncian a continuación.

- **Modelar antes de codificar.** El rediseño de junio de 2026 que aplanó los documentos (eliminación de los submodelos *Tema*, *Subtema*, *Bibliografía*, *CriterioEvaluacion*) enseñó la importancia de contar con la plantilla oficial *antes* de estructurar el esquema. Ante la ausencia de plantilla normativa, un contenido textual libre resultó más flexible y evitó imponer una estructura que la División no había validado.
- **Separar dominio en apps.** Django facilita la separación de dominios en apps independientes con sus propios modelos, vistas, urls y migraciones. Esta disciplina probó ser útil para acotar el impacto de los cambios y para orientar el sistema de pruebas.
- **JWT con *refresh* rotativo.** La combinación *access* corto + *refresh* rotativo mitiga bien el riesgo de filtración de tokens; el precio es que el cliente debe implementar la lógica de *refresh* transparente, lo cual se resolvió con un *interceptor* de *axios*.

- **Testing como red de seguridad.** Las cerca de 4 200 líneas de tests en el backend permitieron refactorizar sin miedo el módulo de autoevaluación en nueve incrementos sucesivos (PR1–PR9), incluyendo cambios profundos como el paso a *scoring* ponderado y el versionado de formularios.
- **Estado del servidor con *TanStack Query*.** Usar una biblioteca dedicada al estado del servidor eliminó una capa entera de código de *fetching* manual y regaló funcionalidades no triviales: *cache*, revalidación, *polling* y *stale-while-revalidate*.

9.3. Trabajo futuro

El sistema entregado es funcional y suficiente para arrancar la operación real de la División, pero se identifican las siguientes líneas de trabajo futuro:

1. **Exportación nativa a PDF.** Incorporar *WeasyPrint* o *ReportLab* en el backend para generar PDFs firmables de cartas temáticas, requisitos de recuperación y resultados de autoevaluación. Esta funcionalidad se prometió en la propuesta y quedó pendiente.
2. **Reset de contraseña autoservicio.** Añadir un flujo de *forgot password* con envío de token único al correo institucional, evitando que el administrador sea el único capaz de restablecer contraseñas.
3. **Integración con directorio institucional.** Si la División cuenta con un LDAP o SSO institucional, integrarlo evitaría mantener contraseñas independientes.
4. **Cobertura de pruebas del frontend.** El frontend actualmente carece de suite de pruebas; añadir Vitest con *React Testing Library* para cubrir al menos los flujos críticos (*login*, *guards* de rol, envío de autoevaluación) elevaría la confianza para futuros cambios.
5. **Estructuración opcional del contenido de cartas.** Si en el futuro la Coordinación de CyAD define una plantilla oficial de carta temática con submodelos (temario por unidad, bibliografía por tipo, criterios de evaluación tabulados), es posible reintroducir la jerarquía sin afectar los documentos existentes mediante una migración adicional.
6. **Notificaciones.** Enviar recordatorios automáticos por correo a profesores que no han publicado cartas o respondido su autoevaluación cerca del cierre del periodo.
7. **Despliegue en el servidor institucional `cyad.azc.uam.mx`.** La puesta en producción está prevista para el trimestre siguiente al reporte, con *Gunicorn* + *Nginx* y respaldos periódicos de PostgreSQL.

En conjunto, el sistema desarrollado sienta una base sólida para la modernización de la administración académica de la División de CyAD y demuestra que un equipo pequeño puede sustituir un flujo manual con *Google Forms* por una solución propia, mantenible y con espacio de crecimiento.

A. Código fuente

Este apéndice presenta *fragmentos curados* del código fuente que ilustran las decisiones de diseño más importantes. El código fuente completo (backend, frontend, tests, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip` de la carpeta de entrega, generado con `git archive HEAD`.

A.1. Estructura del repositorio

```

proyecto_terminal_cyad/
|-- backend/
|   |-- config/           # settings, urls, wsgi
|   |-- core/            # utilidades: pagination, exceptions
|   |-- accounts/        # Usuario, Profesor, JWT
|   |-- catalogos/       # Departamento, Licenciatura, UEA, Periodo
|   |-- documentos/      # CartaTematica, RequisitoRecuperacion
|   |-- autoevaluacion/  # Formulario, Pregunta, Respuesta, Nivel
|   |-- reportes/        # Dashboards y cumplimiento
|   |-- scripts/         # seed_users, seed_demo, create_db
|   |-- tests/           # ~4200 lineas de tests con pytest
|   |-- manage.py
|   |-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- api/          # clientes HTTP (auth, documentos, ...)
|   |   |-- auth/         # AuthContext, ProtectedRoute, RoleRoute
|   |   |-- components/ui/ # Button, Alert, Badge, Table, Modal
|   |   |-- layouts/      # AdminLayout, ProfesorLayout
|   |   |-- pages/
|   |   |   |-- admin/    # dashboards y CRUD por catalogo
|   |   |   |-- profesor/ # cartas, requisitos, autoevaluacion
|   |   |   |-- publico/  # vista Explorar y documentos publicados
|   |   |-- App.jsx       # rutas + guards
|   |   |-- main.jsx      # bootstrap React + TanStack Query
|   |-- index.html
|   |-- package.json
|   |-- vite.config.js
|-- docs/
|   |-- reporte/          # este documento
|   |-- manuales/        # instalacion y usuario
|-- README.md

```

A.2. Backend — Modelo de Usuario con roles

Fragmento de `backend/accounts/models.py`: modelo de usuario personalizado con dos roles.

```
class Usuario(AbstractBaseUser, PermissionsMixin):
    class Rol(models.TextChoices):
        ADMIN      = "ADMIN",      "Administrador"
        PROFESOR   = "PROFESOR",    "Profesor"

    email          = models.EmailField(unique=True)
    nombre         = models.CharField(max_length=150)
    rol            = models.CharField(max_length=10, choices=Rol.choices)
    is_active      = models.BooleanField(default=True)
    is_staff       = models.BooleanField(default=False)

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["nombre"]

    objects = UsuarioManager()
```

A.3. Backend — JWT y throttling

Fragmento de backend/config/settings.py.

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
    "DEFAULT_THROTTLE_CLASSES": (
        "rest_framework.throttling.ScopedRateThrottle",
    ),
    "DEFAULT_THROTTLE_RATES": {
        "login": "5/min",
    },
    "PAGE_SIZE": 20,
}

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=60),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "ROTATE_REFRESH_TOKENS": True,
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

A.4. Backend — Documento academico base

Fragmento de backend/documentos/models.py. Clase abstracta con campos comunes y *snapshot* de identidad del profesor.

```
class DocumentoAcademicoBase(TimeStampedModel):
    class Estado(models.TextChoices):
        BORRADOR = "BORRADOR", "Borrador"
        PUBLICADO = "PUBLICADO", "Publicado"

    profesor = models.ForeignKey(
        Profesor, on_delete=models.SET_NULL, null=True, blank=True
    )
    profesor_nombre = models.CharField(max_length=200, blank=True)
    profesor_correo = models.EmailField(blank=True)

    uea = models.ForeignKey(UEA, on_delete=models.PROTECT)
    periodo = models.ForeignKey(Periodo, on_delete=models.PROTECT)
    nombre_grupo = models.CharField(max_length=50)
    id_grupo = models.CharField(max_length=50)
    horario = models.CharField(max_length=200, blank=True)
    modalidad = models.CharField(
        max_length=15,
        choices=(("PRESENCIAL", "Presencial"), ("REMOTO", "Remoto"), ("MIXTO", "Mixto"))
    )
    estado = models.CharField(
        max_length=15,
        choices=Estado.choices,
        default=Estado.BORRADOR,
    )

    class Meta:
        abstract = True
```

A.5. Backend — Scoring ponderado de autoevaluacion

Fragmento simplificado de backend/autoevaluacion/views.py: calculo del puntaje ponderado por seccion y asignacion del nivel de desempeno.

```
def _calcular_puntaje(respuesta):
    total_pct = Decimal("0")
    for seccion in respuesta.formulario.secciones.all():
        peso = seccion.peso # decimal, suma 100 entre secciones
        obt, maxp = 0, 0
        for pregunta in seccion.preguntas.all():
            resp = respuesta.respuestas_pregunta.filter(pregunta=pregunta).first()
            if resp:
```

```

        obt += sum(o.puntos for o in resp.opciones.all())
        maxp += max((o.puntos for o in pregunta.opciones.all()), default=0)
    if maxp:
        total_pct += peso * Decimal(obt) / Decimal(maxp)
    RespuestaSeccion.objects.update_or_create(
        respuesta=respuesta, seccion=seccion,
        defaults={"puntaje": obt, "puntaje_maximo": maxp},
    )
    respuesta.porcentaje = total_pct
    respuesta.save()

def _asignar_nivel(respuesta):
    nivel = NivelDesempeno.objects.filter(
        formulario=respuesta.formulario,
        porcentaje_min__lte=respuesta.porcentaje,
        porcentaje_max__gte=respuesta.porcentaje,
    ).first()
    respuesta.nivel_desempeno = nivel
    respuesta.save()

```

A.6. Backend — Permisos por rol

Fragmento del sistema de permisos que garantiza que cada profesor solo edite sus documentos.

```

class IsOwnerProfesorOrAdminReadOnly(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated

    def has_object_permission(self, request, view, obj):
        if request.user.rol == Usuario.Rol.ADMIN:
            return request.method in permissions.SAFE_METHODS
        return obj.profesor and obj.profesor.usuario_id == request.user.id

```

A.7. Frontend — Cliente HTTP con refresh automatico de JWT

Fragmento de frontend/src/api/client.js.

```

const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

client.interceptors.request.use((config) => {
  const token = sessionStorage.getAccessToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

```



```

let refreshing = null;

client.interceptors.response.use(
  (r) => r,
  async (error) => {
    const { config, response } = error;
    if (response?.status === 401 && !config.__retried) {
      config.__retried = true;
      refreshing = refreshing ?? refreshAccessToken();
      const newToken = await refreshing.finally(() => (refreshing = null));
      if (newToken) {
        config.headers.Authorization = `Bearer ${newToken}`;
        return client(config);
      }
    }
    return Promise.reject(error);
  }
);

```

A.8. Frontend — Guard de rutas por rol

Fragmento de frontend/src/auth/RoleRoute.jsx.

```

export function RoleRoute({ role, children }) {
  const { user, isLoading } = useAuth();
  if (isLoading) return <Loading />;
  if (!user) return <Navigate to="/login" replace />;
  if (user.rol !== role) return <Navigate to="/" replace />;
  return children;
}

export const AdminRoute = ({ children }) =>
  <RoleRoute role="ADMIN" >{children}</RoleRoute>;
export const ProfesorRoute = ({ children }) =>
  <RoleRoute role="PROFESOR" >{children}</RoleRoute>;

```

A.9. Frontend — Renderizador dinamico de preguntas

Fragmento de frontend/src/pages/profesor/autoevaluacion/AutoevaluacionFormPa que despacha por tipo de pregunta.

```

function PreguntaRenderer({ pregunta, value, onChange }) {
  switch (pregunta.tipo) {
    case "TEXTO_CORTO":
      return <TextInput {...{value, onChange}} />;
    case "TEXTO_LARGO":
      return <TextArea {...{value, onChange}} />;
  }
}

```

```

        case "OPCION_UNICA":      return <RadioGroup opciones={pregunta.opciones} {...
        case "CASILLAS":         return <CheckGroup opciones={pregunta.opciones} {...
        case "SI_NO":            return <YesNo      {...{value, onChange}} />;
        case "ESCALA_LINEAL":    return <LinearScale config={pregunta.config} {...{va
        case "LISTA_DESPLEGABLE": return <Select    opciones={pregunta.opciones} {...
        case "CUADRICULA":       return <Grid       config={pregunta.config} filas={p
        default:                 return <UnknownType tipo={pregunta.tipo} />;
    }
}

```

A.10. Backend — Comando de importacion CSV para UEA

Comando de gestion `python manage.py cargar_catalogos_csv` para carga masiva idempotente. Fragmento simplificado.

```

class Command(BaseCommand):
    help = "Carga catalogos (Areas, Licenciaturas, Posgrados, UEAs) desde CSV"

    def add_arguments(self, parser):
        parser.add_argument("--ueas", type=str, help="Path al CSV de UEAs")

    def handle(self, *args, **opts):
        ruta = opts["ueas"]
        with open(ruta, newline="", encoding="utf-8") as fh:
            for fila in csv.DictReader(fh):
                UEA.objects.update_or_create(
                    clave=fila["clave"],
                    licenciatura=Licenciatura.objects.get(clave=fila["licenciatura"],
                    defaults={
                        "nombre":      fila["nombre"],
                        "trimestre": int(fila["trimestre"]),
                        "creditos":  int(fila["creditos"]),
                        "tipo":        fila["tipo"],
                    },
                )

```

A.11. Nota sobre el listado completo

Los archivos aquí incluidos son una selección representativa de los fragmentos más ilustrativos. El código fuente completo (backend, frontend, *tests*, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip`. Para regenerarlo desde el repositorio se ejecuta desde la raíz del proyecto:

```
git archive HEAD -o docs/reporte/entregables/codigo_fuente.zip
```

Adicionalmente, la documentación viva de la API se genera automáticamente vía *drf-spectacular* y se sirve en `/api/docs/` (interfaz Swagger UI) y `/api/schema/` (esquema OpenAPI en YAML).

B. Entregables del plan de trabajo

La Tabla 5 replica el calendario de actividades comprometido en la propuesta (198 horas en total para el Trimestre 2026-Primavera) con el estado real de cada entregable al cierre del proyecto.

Tabla 5. Entregables prometidos en la propuesta y su estado real.

No.	Actividad	Hrs.	Estado y evidencia
1	Diseño e implementación de la base de datos	20	Completo. 21 migraciones, seis apps, PostgreSQL 16. Diagrama ER en Figura 3.
2	Diseño de las interfaces por módulo	10	Completo. UI implementada en React 19 + Tailwind, con componentes reutilizables (Button, Alert, Badge, Table, Modal, FormField, etc). Capturas en Sección 7.
3	Desarrollo del módulo de autenticación	25	Completo. accounts/ + 491 líneas de tests (tests/test_auth.py). Login JWT con <i>throttling</i> .
4	Desarrollo del módulo de cartas temáticas	25	Completo. documentos/ (parte CartaTematica) + tests en tests/test_documentos.py. Sin exportación PDF/DOCX (véase Sección 8).
5	Desarrollo del módulo de requisitos de recuperación	25	Completo. documentos/ (parte RequisitoRecuperacion). Mismo alcance/limitaciones que cartas.
6	Desarrollo del módulo de autoevaluación docente	25	Ampliado. autoevaluacion/ con <i>form-builder</i> , 8 tipos de pregunta, <i>scoring</i> ponderado y versionado (9 incrementos PR1–PR9). 2 044 líneas de tests.
7	Desarrollo del módulo de generación de reportes	25	Completo. reportes/ + 390 líneas de tests. Export XLSX en cliente (no PDF).
8	Integración del sistema con la base de datos	10	Completo. ORM operativo, <i>fixtures</i> , comando <code>cargar_catalogos_csv</code> y <i>scripts</i> <code>seed_users</code> / <code>seed_demo</code> .
9	Prueba y corrección de errores	8	Completo. ≈4 186 líneas de tests + <i>smoke test</i> end-to-end. Historial git con <i>fixes</i> rastreables (por ejemplo <i>fix(seguridad)</i> , <i>fix(autoevaluacion)</i> , <i>fix(catalogos)</i>).

No.	Actividad	Hrs.	Estado y evidencia
10	Documentación para usuarios	10	Completo. docs/manuales/usuario.tex y docs/manuales/instalacion.tex.
11	Redacción del reporte final del proyecto	15	Completo. Este documento (docs/reporte/main.pdf) con nueve secciones, dos anexos, índices y bibliografía.
<i>Entregables adicionales no contemplados originalmente:</i>			
E1	Vista pública de exploración (cascada periodo→programa→UEA)	—	Entregado (extra). Beneficia a estudiantes al elegir carga.
E2	Catálogos administrables completos + importación CSV	—	Entregado (extra). Departamentos, licenciaturas, posgrados, áreas, UEA, periodos.
E3	Suite de pruebas de ~4 200 líneas en backend	—	Entregado (extra). Ver backend/tests/.
E4	Documentación OpenAPI/Swagger auto-generada	—	Entregado (extra). Vía <i>drf-spectacular</i> en /api/schema/ y /api/docs/.

Adicionalmente, la carpeta digital de entrega a la Coordinación de Estudios contiene:

- `reporte_final.pdf` — compilación de este documento, sin restricciones.
- `codigo_fuente.zip` — generado con `git archive HEAD`, descomprimible sin contraseña.
- `manual_usuario.pdf` y `manual_instalacion.pdf`.
- `cobertura/` — reporte HTML de cobertura de tests generado con `pytest -cov`.

Las/Los asesoras(es) se responsabilizan de guiar al alumno y de que todos los recursos mencionados en la Factibilidad Técnica estarán disponibles para el alumno, de modo que el proyecto de integración se pueda concluir en tiempo y forma.

M. en C Cesar Benavides Alvarez

M. en C Josué Figueroa González

Referencias

- [1] H. P. Leyva López, M. G. Pérez Vera, and S. M. Pérez Vera, "Google forms en la evaluación diagnóstica como apoyo en las actividades docentes. caso con estudiantes de la licenciatura en turismo," *RIDE. Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, vol. 9, no. 17, pp. 84–111, 2018.
- [2] J. M. A. Echeverría, "La autoevaluación docente de aula: un camino para mejorar la práctica educativa," *Revista Electrónica Diálogos Educativos. REDE*, vol. 11, no. 22, pp. 183–196, 2011.
- [3] A. Cuautle Vasquez, "Sistema para la configuración de formularios y captura de los resultados de la evaluación de atributos de egreso en un proceso de mejora continua," Proyecto terminal, División de Ciencias Básicas e Ingeniería, Universidad Autónoma Metropolitana Azcapotzalco, México, 2022.
- [4] G. Callisaya and A. Emerson, "Sistema académico de gestión escolar trimestralizado," *Computer*, 2024. [Online]. Available: <https://repositorio.upea.bo/jspui/handle/123456789/1062>
- [5] E. M. Rojas, M. R. Ramírez, H. B. R. Moreno, M. d. C. S. Soto, C. O. Millán, Nora d., and L. M. C. Suarez, "Sistema de gestión académica a través del desarrollo de modelo-vista-controlador," *Computer*, vol. 20, no. 9, pp. 1083–1093, 01 2019. [Online]. Available: <https://www.proquest.com/scholarly-journals/sistema-de-gestiÃşn-acadÃşmica-travÃşs-del/docview/2195127267/se-2>
- [6] J. C. González *et al.*, "Sistema de gestión administrativa para la escuela secundaria n° 44 "jorge luis borges"," B.S. thesis, Facultad de Tecnología y Ciencias Aplicadas. Departamento de Informática., 2017.
- [7] M. G. P. Martínez, "La digitalización de las instituciones de educación superior en el salvador: Retos y oportunidades," *Perspectiva Científica*, vol. 1, no. 2, pp. 27–32, 2024.
- [8] G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in smalltalk-80," *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988.

- [9] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [10] Django Software Foundation, "Django documentation," 2024. [Online]. Available: <https://docs.djangoproject.com/en/5.1/>
- [11] T. Christie and Django REST Framework contributors, "Django REST Framework documentation," 2024. [Online]. Available: <https://www.django-rest-framework.org/>
- [12] PostgreSQL Global Development Group, "PostgreSQL 16 documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [13] Meta Open Source, "React documentation," 2024. [Online]. Available: <https://react.dev/>
- [14] Vite contributors, "Vite: Next generation frontend tooling," 2024. [Online]. Available: <https://vitejs.dev/>
- [15] Tailwind Labs, "Tailwind CSS: A utility-first css framework," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [17] E. Sánchez and R. Villalobos, "Autenticación y autorización basadas en tokens para aplicaciones web modernas," *Research in Computing Science*, vol. 148, no. 5, pp. 103–112, 2019.
- [18] OWASP Foundation, "OWASP Top 10: The ten most critical web application security risks," 2021. [Online]. Available: <https://owasp.org/Top10/>

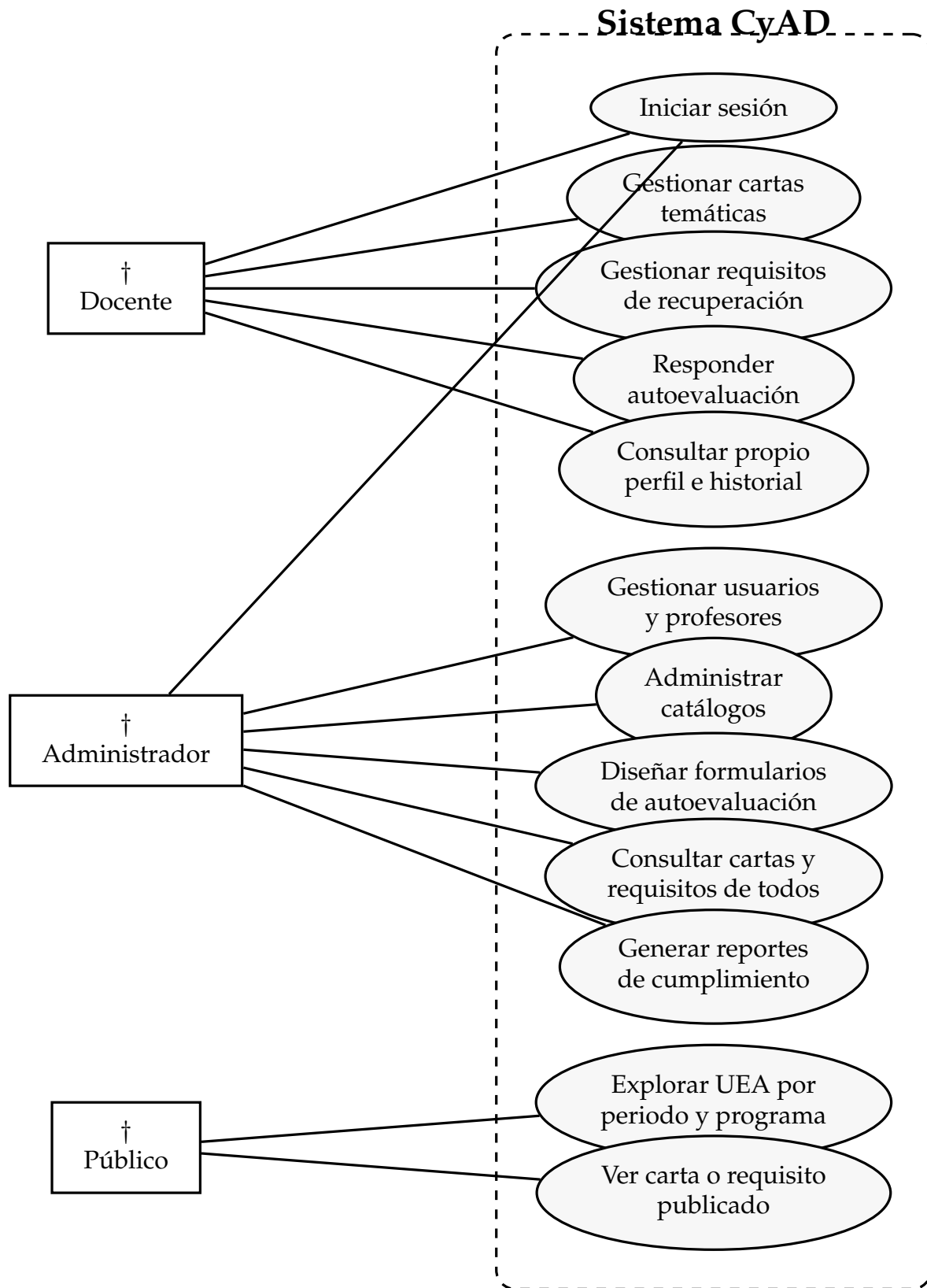


Figura 2. Diagrama de casos de uso del sistema.

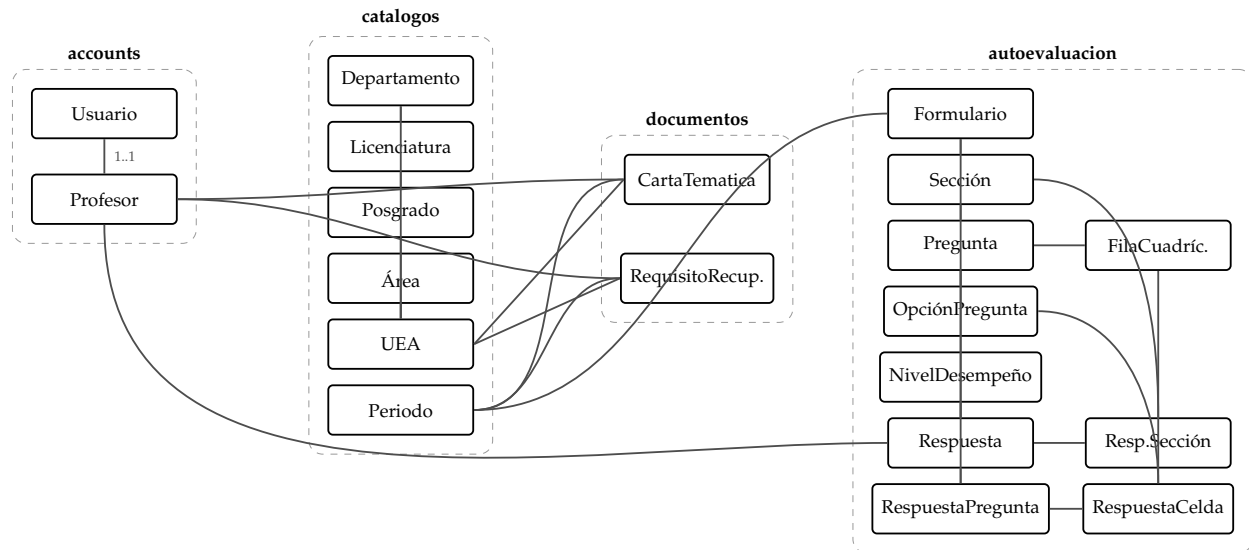


Figura 3. Diagrama Entidad-Relación del sistema (agrupado por app Django).

La pantalla de inicio de sesión muestra el logo **CYAD - UAM-A** y el título **Sistema de Proyecto Terminal**. Debajo del título, se indica "Inicia sesión con tu cuenta institucional".

El formulario de inicio de sesión incluye:

- Un campo de texto para el **Correo electrónico *** con el valor predeterminado `usuario@uam.mx`.
- Un campo de texto para la **Contraseña *** con caracteres ocultos por puntos.
- Un botón azul con el texto **Iniciar sesión**.

Debajo del formulario, hay un enlace que dice [← Volver a documentos públicos](#) y el pie de página indica: **División de Ciencias y Artes para el Diseño - UAM Azcapotzalco**.

Figura 4. Pantalla de inicio de sesión.

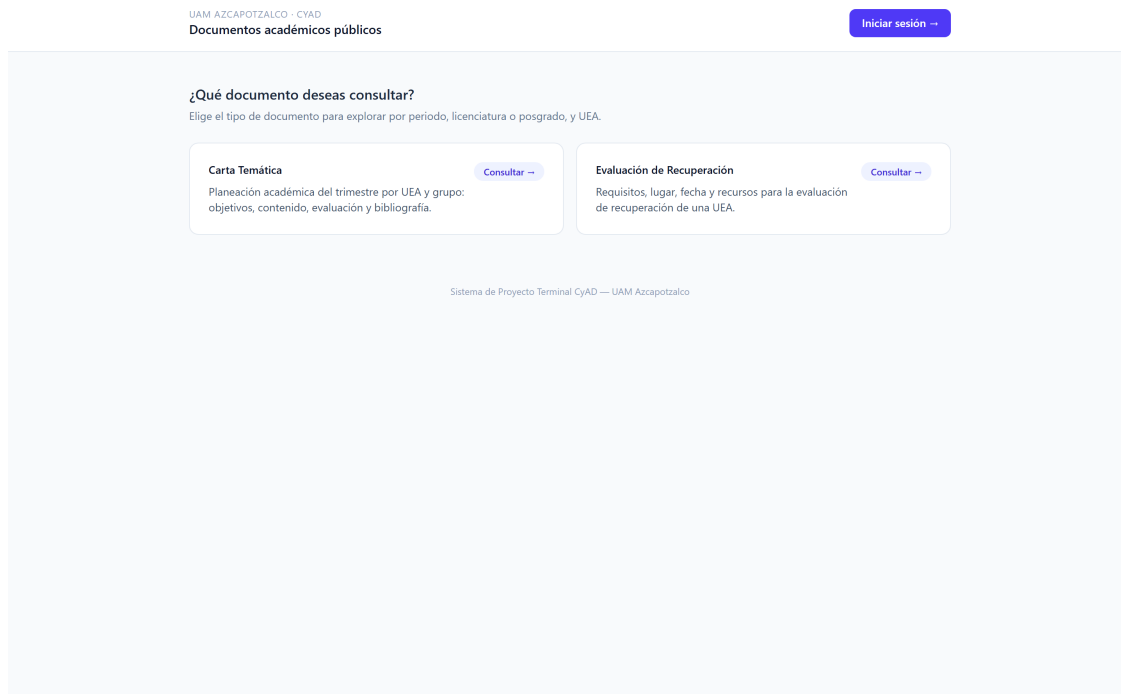


Figura 5. *Dashboard* del profesor: tarjetas de resumen y listas agrupadas por periodo.

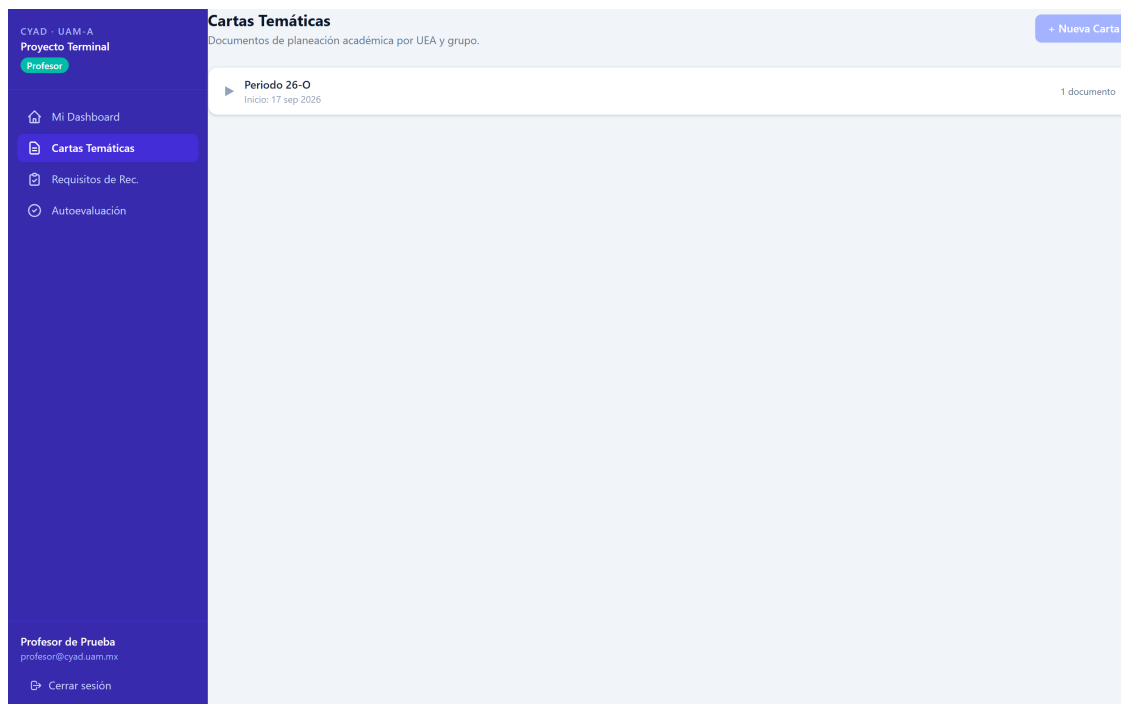


Figura 6. Lista de cartas temáticas del profesor.

CYAD - UAM-A
Proyecto Terminal
Profesor

Mi Dashboard
Cartas Temáticas
Requisitos de Rec.
Autoevaluación

Profesor de Prueba
profesor@cyad.uam.mx
Cerrar sesión

Nueva Carta Temática

Selección de UEA
Búscala por clave o nombre y selecciónala de la lista.

Buscar UEA *
ej. 1100023 o Diseño Industrial
Escribe parte de la clave o del nombre y selecciona una opción.

Información del grupo

Periodo (asignado automáticamente)
Sin periodo activo para Cartas Temáticas. Contacta al administrador.

Modalidad
-- Opcional --

Nombre del grupo *
ej. Grupo A

ID del grupo *
ej. 2026-I-A1

Horario *
ej. Lun-Mié 9:00-11:00

Contenido
Todos los campos aceptan texto libre. Los que dejes vacíos no se mostrarán en la vista pública.

Descripción de la UEA
Descripción de la UEA
Breve descripción general de la UEA.

Figura 7. Formulario de creación/edición de carta temática.

CYAD - UAM-A
Proyecto Terminal
Profesor

Mi Dashboard
Cartas Temáticas
Requisitos de Rec.
Autoevaluación

Profesor de Prueba
profesor@cyad.uam.mx
Cerrar sesión

Nuevo Requisito de Recuperación

Selección de UEA
Búscala por clave o nombre y selecciónala de la lista.

Buscar UEA *
ej. 1100023 o Diseño Industrial
Escribe parte de la clave o del nombre y selecciona una opción.

Información del grupo

Periodo (asignado automáticamente)
Sin periodo activo para Requisitos. Contacta al administrador.

Modalidad de la UEA
-- Opcional --

Nombre del grupo *
ej. Grupo A

ID del grupo *
ej. 2026-I-A1

Horario *
ej. Lun-Mié 9:00-11:00

Detalles de la evaluación

Lugar
ej. Aula H-204 / Liga Zoom: https://... contraseña: 1234
Texto libre. Si es remoto, incluye liga y contraseña.

Duración aproximada
ej. 2 horas

Fecha y hora
ej. Lunes 15 de mayo, 10:00 h

Figura 8. Formulario de requisitos de recuperación.

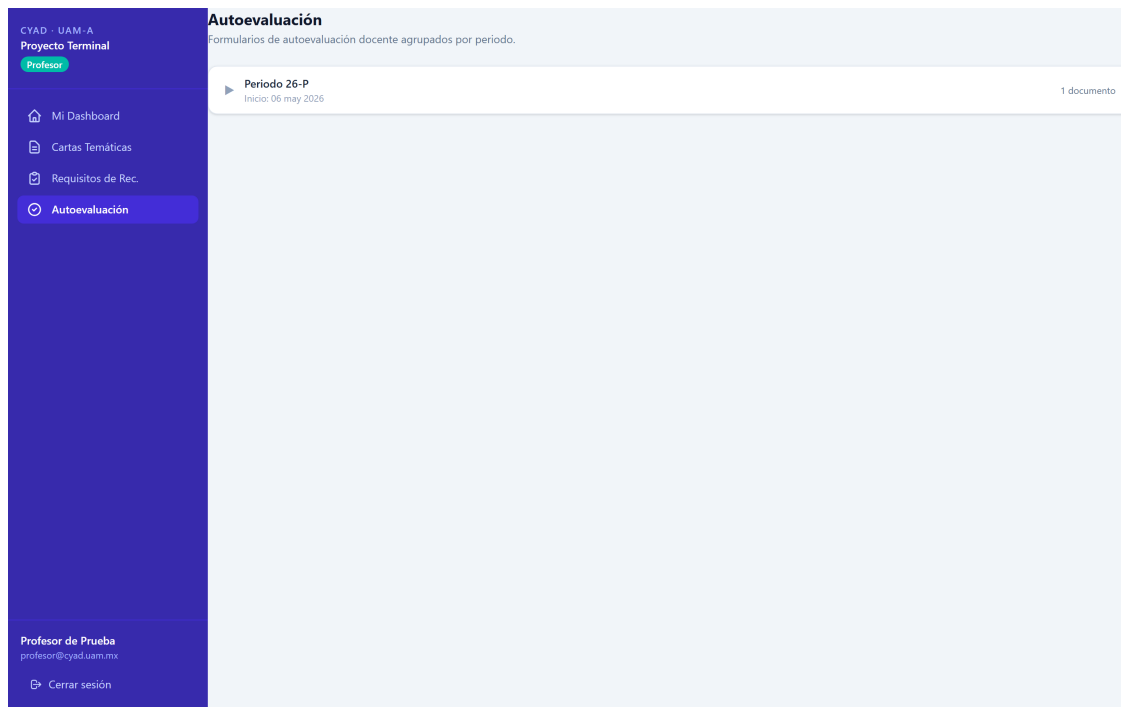


Figura 9. Formulario de autoevaluación con renderizado dinámico de preguntas.

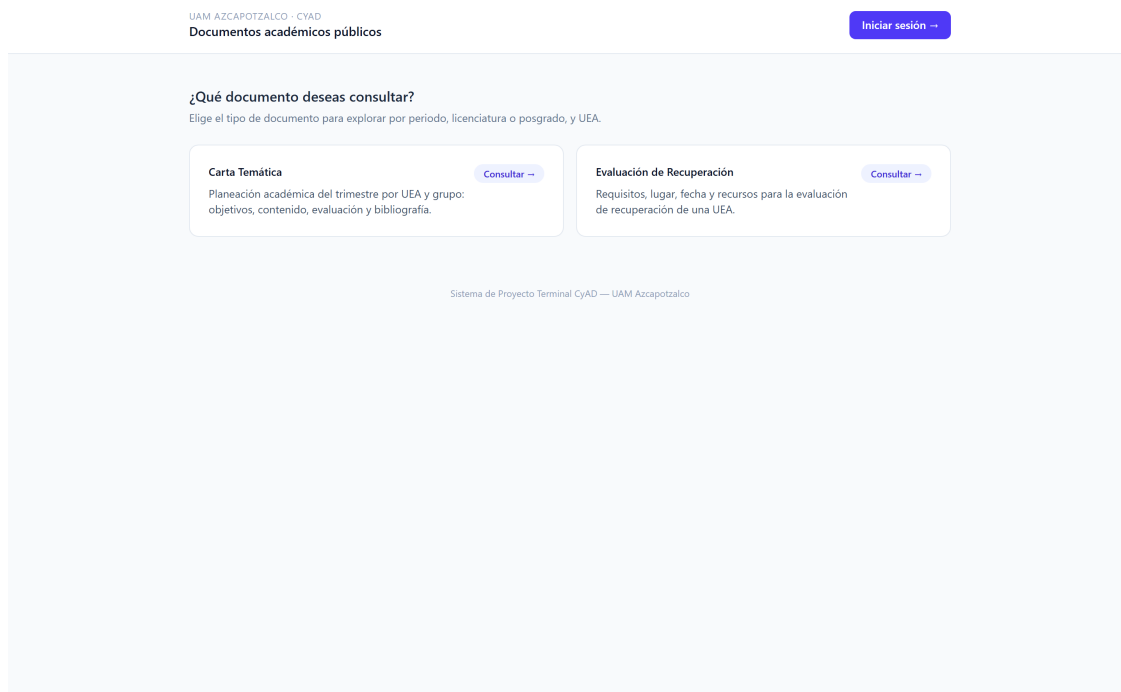


Figura 10. *Dashboard* del administrador: KPIs globales y cumplimiento por departamento.

Profesores

Buscar por nombre, correo o N° económico...

PROFESOR	N° ECONÓMICO	DEPARTAMENTO	EMAIL DE ACCESO	ESTADO			
ABAD SÁNCHEZ ANTONIO RODRIGO ares@azc.uam.mx	4968	Procesos y Técnicas de Realización	ares@azc.uam.mx	ACTIVO	Editar	Contraseña	Eliminar
Cesar Benavides Alvarez cesarbenavides@azc.uam.mx	12345	Evaluación del Diseño en el Tiempo	cesarbenavides@azc.uam.mx	ACTIVO	Editar	Contraseña	Eliminar
Javier Ortega Rios jortega@azc.uam.mx	30245	Investigación y Conocimiento para el Diseño	profesor2@cyad.uam.mx	ACTIVO	Editar	Contraseña	Eliminar
Profesor de Prueba profesor@cyad.uam.mx	11133	Evaluación del Diseño en el Tiempo	profesor@cyad.uam.mx	ACTIVO	Editar	Contraseña	Eliminar

Figura 11. Gestión de profesores: crear, editar, restablecer contraseña.

Autoevaluación

Formularios de evaluación docente.

+ Nuevo Formulario

FORMULARIO	ESTADO	PREGUNTAS	RESPUESTAS				
Autoevaluación Docente 26-I 26-I	BORRADOR	6	0	Ver	Editar	Duplicar	Eliminar
Autoevaluación Docente 26-P completo 26-P	BORRADOR	41	0	Ver	Editar	Duplicar	Eliminar
Autoevaluación Docente 26-O 26-O	PUBLICADO	24	1	Ver	Editar	Duplicar	Eliminar
Autoevaluación Docente 26-P 26-P	PUBLICADO	24	3	Ver	Editar	Duplicar	Eliminar

Figura 12. Constructor de formularios de autoevaluación (*form-builder*).

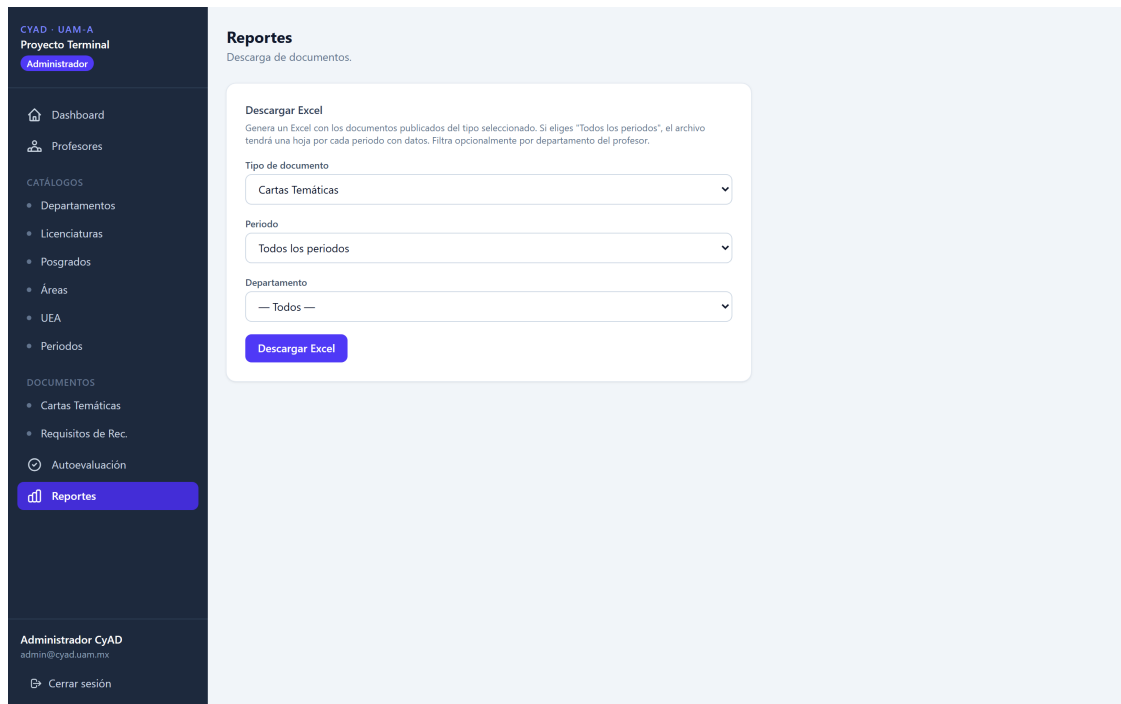


Figura 13. Panel de reportes con descarga en XLSX.

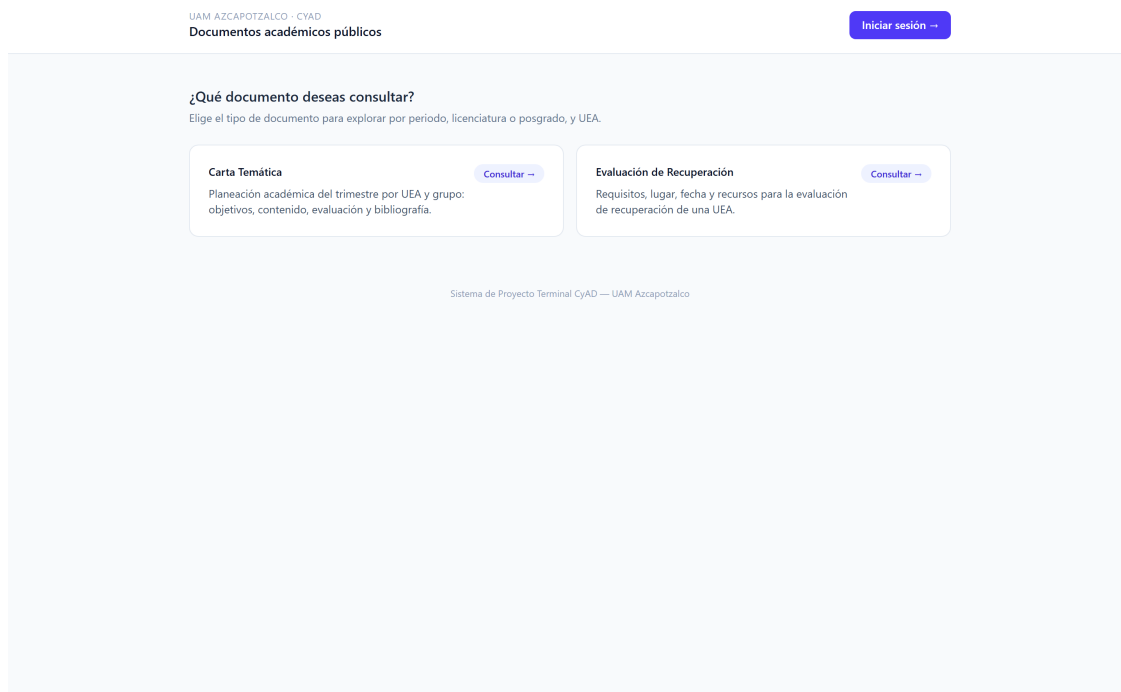
Figura 14. Vista pública *Explorar*: cascada periodo→programa→UEA con acordeón de documentos.



Figura 15. Vista pública de una carta temática publicada, con formato imprimible.